

# Meta-analysis of Anorexia Nervosa microbiome datasets with SIAMCAT

2026-04-13

## Contents

|          |                                                       |           |
|----------|-------------------------------------------------------|-----------|
| <b>1</b> | <b>Data Preprocessing</b>                             | <b>1</b>  |
| 1.1      | Load & tidy per-study data . . . . .                  | 1         |
| 1.2      | Build per-study SIAMCAT objects . . . . .             | 4         |
| <b>2</b> | <b>Univariate association testing</b>                 | <b>5</b>  |
| <b>3</b> | <b>Merge for multivariate modelling and ML</b>        | <b>9</b>  |
| <b>4</b> | <b>Building cross-validated LASSO models</b>          | <b>10</b> |
| <b>5</b> | <b>LOSO coefficients for diagnostics (raw values)</b> | <b>21</b> |
| <b>6</b> | <b>Meta-Analysis with Linear Mixed-Effect Models</b>  | <b>26</b> |
| <b>7</b> | <b>Descriptive Analysis</b>                           | <b>33</b> |

## 1 Data Preprocessing

### 1.1 Load & tidy per-study data

Rarefied to 25000 for descriptive analyses

```
# Rarefied to 25000 for descriptive analyses

rarefaction_depth <- 25000
##Borgo 2017 -----
# 15 ANT1 vs 15 HC
borgo_IDs <- read_csv(here(borgo_dir, 'SraRunTable.csv')) |>
  select(Run, `Sample Name`) |>
  rename(sample_id = 'Sample Name',
         run = 'Run')
borgo_meta <- read_csv(here(borgo_dir, "borgo2017meta.csv")) |>
  janitor::clean_names() |>
  # add new ID pairs (correct, 01.09)
  select(-run) |>
  left_join(borgo_IDs) |>
  select(run, everything()) |>
  # continue
  mutate(group = if_else(str_detect(sample_id, 'AN'),
                        "ANT1", "HC")) |>
  mutate(study='borgo2017') |>
  column_to_rownames("run")
```

```

#add bmi_quant to meta
borgo_meta_bmi_quant <- borgo_meta %>%
  mutate(bmi_quant = if_else(group == "HC", 0, max(as.numeric(bmi_kg_m2)[!is.na(as.numeric(bmi_kg_m2))]))

borgo_counts <- readRDS(here(borgo_dir, "res_idtaxa_genus.rds")) |>
  phyloseq::otu_table(taxa_are_rows = TRUE)

borgo_RA <- relab_transfo(borgo_counts)

borgo_rarefied <- borgo_counts |>
  phyloseq::rarefy_even_depth(sample.size=rarefaction_depth,
                              rngseed = 1,
                              replace=FALSE)
borgo_rarefied_RA <- relab_transfo(borgo_rarefied)

## Mack 2016 -----
# counts - median 71K reads...
# 55 ANT1 vs 54 HC
mack_ids <- readxl::read_xlsx(here(mack_dir, 'Metadaten-Mack-neu.xlsx'))
mack_meta <- read_csv(here(mack_dir, "mack2016meta.csv")) |>
  janitor::clean_names() |>
  left_join(mack_ids) |>
  mutate(group = case_when(
    str_detect(anonymized_name, 'ANT1') ~ 'ANT1',
    str_detect(anonymized_name, 'HC') ~ 'HC',
    str_detect(anonymized_name, 'QC') ~ NA,
    TRUE ~ NA)) |>
  filter(!is.na(group)) |>
  mutate(study='mack2016') |>
  column_to_rownames("run")

mack_bmi_hc <- readxl::read_xlsx(here(mack_dir, "Metadata_nachHeidelberg_Mai17_2024.xlsx"))[,1:6] %>%
  as_tibble() %>%
  janitor::clean_names()
mack_bmi_hc <- mack_bmi_hc %>%
  mutate(
    numeric_part = str_extract(sample_id, "\\d+"),
    stripped_part = strip_leading_zeros(numeric_part),
    anonymized_name = paste0(timepoint, "_", stripped_part)
  ) %>%
  dplyr::select(-numeric_part, -stripped_part) %>% # Remove intermediate columns
  subset(., patient == "HC")

#create bmi_quant for all mack data:
mack_meta_full_bmi <- mack_meta %>%
  rownames_to_column("run_id") %>%
  left_join(mack_bmi_hc %>% select(anonymized_name, BMI_new = bmi),
            by = "anonymized_name") %>%
  mutate(BMI = if_else(group == "HC" & is.na(BMI), BMI_new, BMI)) %>%
  select(-BMI_new) %>%
  column_to_rownames("run_id")

```

```

mack_meta_full_bmi <- mack_meta_full_bmi %>%
  mutate(bmi_quant = if_else(group == "HC", 0, max(BMI[!is.na(BMI)]) - BMI))

mack_counts <- readRDS(here(mack_dir, "res_idtaxa_genus.rds")) |>
  as.data.frame() |>
  select(all_of(rownames(mack_meta))) |>
  phyloseq::otu_table(taxa_are_rows = TRUE)

mack_RA <- relab_transfo(mack_counts)

mack_rarefied <- mack_counts |>
  phyloseq::rarefy_even_depth(sample.size=rarefaction_depth,
                              rngseed = 1,
                              replace=FALSE)
mack_rarefied_RA <- relab_transfo(mack_rarefied)

## Prochazkova 2020 -----
## 33K median read count
proch_sra <- read_csv(here(proch_dir, 'SraRunTable (5).txt')) |>
  select(Run, BioSample)
proch_meta <- readxl::read_xlsx(here(proch_dir, "Table S5_formated.xlsx")) |>
  janitor::clean_names() |>
  # filter out QC and other samples
  mutate(group = case_when(category == "1" ~ "ANT1",
                           # don't end up using post-hospitalization
                           category == "2" ~ "ANT2",
                           category == 'C' ~ 'HC',
                           TRUE ~ NA)) |>
  filter(group %in% c('ANT1', 'HC')) |>
  # match to SRA table
  mutate(BioSample = str_split(accession_no_bacteriome, ' ') |>
    map_chr(~ tail(., 1))) |>
  mutate(BioSample=str_remove_all(BioSample,fixed('('))) |>
  mutate(BioSample=str_remove_all(BioSample,fixed(')'))) |>
  left_join(proch_sra) |>
  mutate(study='proch2020') |>
  select(Run, group, everything(), -contains('accession')) |>
  column_to_rownames("Run")
#add bmi_quant:
proch_meta_bmi_quant <- proch_meta %>%
  mutate(bmi_quant = if_else(group == "HC", 0, max(as.numeric(bmi_kg_m2)[!is.na(as.numeric(bmi_kg_m2))]))

proch_counts <- readRDS(here(proch_dir, "res_idtaxa_genus.rds")) |>
  as.data.frame() |>
  select(rownames(proch_meta)) |>
  phyloseq::otu_table(taxa_are_rows = TRUE)

proch_RA <- relab_transfo(proch_counts)

proch_rarefied <- proch_counts |>
  phyloseq::rarefy_even_depth(sample.size=rarefaction_depth,
                              rngseed = 1,

```

```

                                replace=FALSE)
proch_rarefied_RA <- relab_transfo(proch_rarefied)

## Monteleone 2021 -----
# metadata is patient-level this is sample-level/dups
# 31K median
monte_counts <- readRDS(here(monte_dir, "res_idtaxa_genus.rds")) |>
  as.data.frame() |>
  rename_all(~ sub("_.*", "", .)) |>
  # prune duplicates/other time points
  select(!matches('A|B')) |>
  phyloseq::otu_table(taxa_are_rows = TRUE)

monte_meta <- readxl::read_xlsx(
  here(monte_dir, "Clinical data anorexia samples.xlsx")) |>
  janitor::clean_names() |>
  filter(time_point=='T0' & id %in% colnames(monte_counts)) |>
  mutate(group = ifelse(diagnosis=='C','HC','ANT1')) |>
  filter(id %in% colnames(monte_counts)) |>
  arrange(id) |>
  mutate(study='monte2021') |>
  select(group,everything(),-diagnosis) |>
  column_to_rownames('id')
#add bmi_quant:
monte_meta_bmi_quant <- monte_meta %>%
  mutate(bmi_quant = if_else(group == "HC", 0, max(as.numeric(bmi)[!is.na(as.numeric(bmi))]) - as.numeric(bmi)))

monte_counts <- monte_counts[,rownames(monte_meta)]
monte_RA <- relab_transfo(monte_counts)

monte_rarefied <- monte_counts |>
  phyloseq::rarefy_even_depth(sample.size=rarefaction_depth,
                              rngseed = 1,
                              replace=FALSE)
monte_rarefied_RA <- relab_transfo(monte_rarefied)

```

Focus on a unified set of features — needed for ML later, and reduces bias of larger studies which have more features sampled/detected not present in the smaller ones

```

common_genera <- Reduce(intersect,
  list(rownames(borgo_RA),
        rownames(mack_RA),
        rownames(proch_RA),
        rownames(monte_RA))) |>
  setdiff("not_resolved")

```

## 1.2 Build per-study SIAMCAT objects

```

# standard preprocessing -- each dataset transformed to relative abundances
# "not resolved" (unmapped genomes) have been removed in previous block
sc_list <- list(
  Borgo = make_siamcat(borgo_RA[common_genera, ],
                      borgo_meta,
                      rel_abun=TRUE),

```

```

Mack    = make_siamcat(mack_RA[common_genera, ],
                      mack_meta,
                      rel_abun=TRUE),
Proch   = make_siamcat(proch_RA[common_genera, ],
                      proch_meta,
                      rel_abun=TRUE),
Monte   = make_siamcat(monte_RA[common_genera, ],
                      monte_meta,
                      rel_abun=TRUE))

```

## 2 Univariate association testing

```

# build association tables for each cohort
assoc_list <- lapply(sc_list, check.associations)

# define recoding
study_map <- c(
  "Borgo" = "Borgo2017",
  "Mack"  = "Mack2016",
  "Monte" = "Monte2021",
  "Proch" = "Proch2020"
)

# reshape to long format
assoc_long <- map2_df(assoc_list, names(assoc_list),
  ~ associations(.x) |>
    as.data.frame() |>
    rownames_to_column("genus") |>
    as_tibble() |>
    mutate(Study = recode(.y, !!!study_map)))

# pick genera with strong single-feature AUROC or FDR 0.01
keepers <- assoc_long |>
  mutate(flag = auc < 0.25 | auc > 0.75 | p.adj <= 0.01) |>
  group_by(genus) |>
  summarise(any_keep = any(flag),
            m_auc = mean(auc),
            .groups = "drop") |>
  filter(any_keep) |>
  arrange(m_auc) |>
  pull(genus)

plot_df <- assoc_long |>
  filter(genus %in% keepers) |>
  mutate(lab = if_else(p.adj < 0.01, "*", ""),
         genus = factor(genus, levels = rev(keepers)))

fplot_assoc <- ggplot(plot_df, aes(Study, genus, fill = fc)) +
  geom_tile() +
  geom_text(aes(label = lab), size = 5) +
  scale_fill_gradient2(low = "#3B6FB6",

```

```

        mid = "white", high = "#D41645",
        limits = c(-2.7, 2.7),
        name = "Generalized\nfold change") +

theme_minimal() +
theme(axis.text.y = element_text(size = 9),
      panel.grid = element_blank()) +
labs(x = NULL, y = NULL)

plot_df <- assoc_long |>
  filter(genus %in% keepers) |>
  mutate(
    lab = case_when(
      p.adj < 0.01 & auc > 0.75 ~ "+",
      p.adj < 0.01 & auc < 0.25 ~ "-*",
      p.adj < 0.01 ~ "*",
      auc > 0.75 ~ "+",
      auc < 0.25 ~ "-",
      TRUE ~ ""
    ),
    genus = factor(genus, levels = rev(keepers))
  )

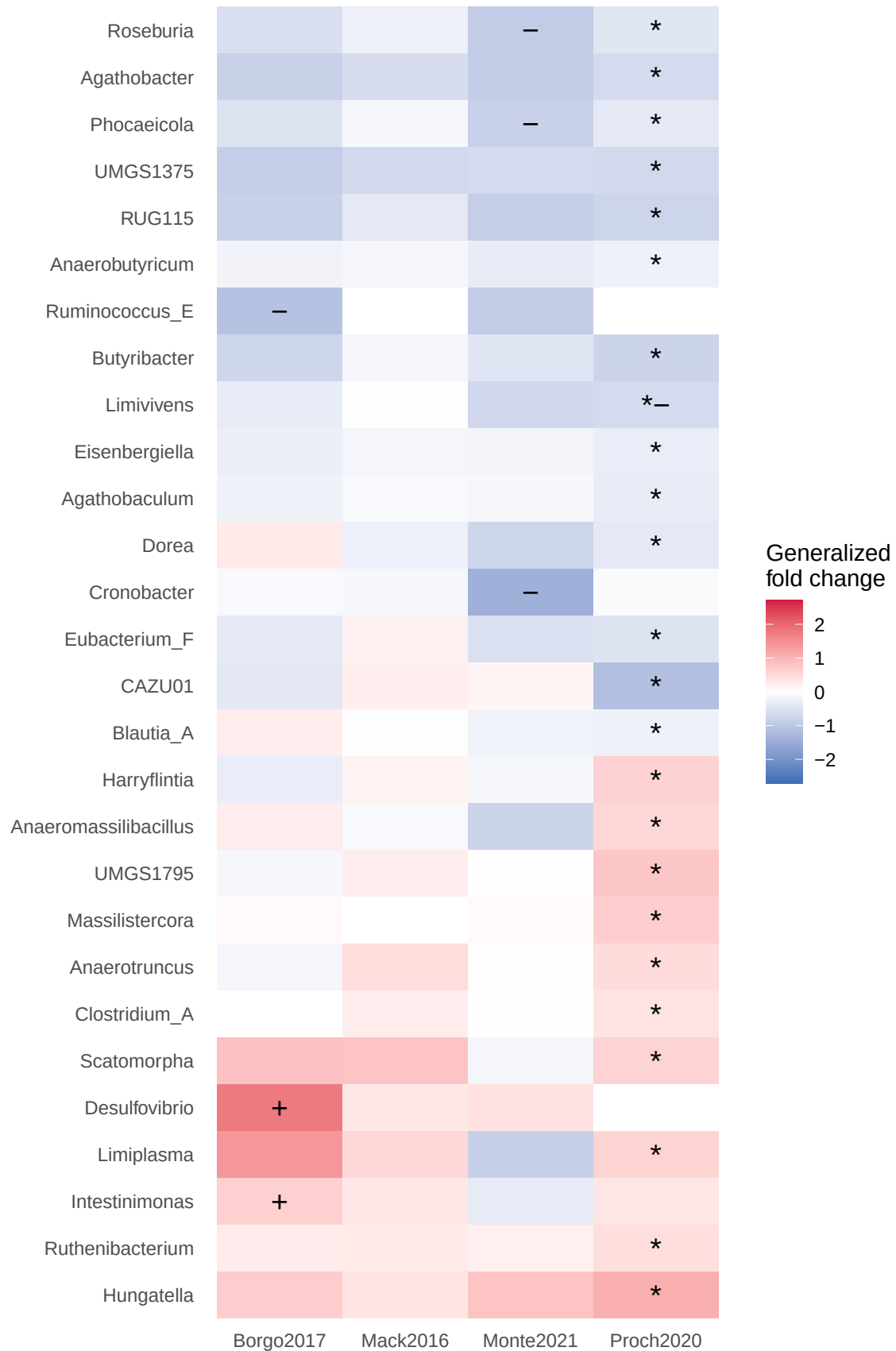
fplot_assoc <- ggplot(plot_df, aes(Study, genus, fill = fc)) +
  geom_tile() +
  geom_text(aes(label = lab), size = 5) +
  scale_fill_gradient2(low = "#3B6FB6",
                      mid = "white", high = "#D41645",
                      limits = c(-2.7, 2.7),
                      name = "Generalized\nfold change") +

  theme_minimal() +
  theme(axis.text.y = element_text(size = 9),
        panel.grid = element_blank()) +
  labs(x = NULL, y = NULL)
plot_df

## # A tibble: 112 x 12
##   genus      fc p.val  auc auc.ci.l auc.ci.h pr.shift pr.n pr.p p.adj
##   <fct>    <dbl> <dbl> <dbl>    <dbl>    <dbl>    <dbl> <dbl> <dbl> <dbl>
## 1 Agathoba~ -0.806  0.0251 0.258  0.0714  0.444  0      1      1  0.734
## 2 Agathoba~ -0.248  0.561  0.436  0.221  0.650 -0.133  1      0.867 0.871
## 3 Anaerobu~ -0.210  0.590  0.44  0.227  0.653 -0.0667 0.933 0.867 0.875
## 4 Anaeroma~  0.251  0.402  0.591  0.379  0.804  0      0.733 0.733 0.806
## 5 Anaerotr~ -0.156  0.883  0.482  0.262  0.702 -0.133  0.8  0.667 1
## 6 Blautia_A  0.244  0.455  0.582  0.358  0.806  0      1      1  0.846
## 7 Butyriba~ -0.731  0.198  0.36  0.147  0.573 -0.2  0.933 0.733 0.768
## 8 CAZU01    -0.398  0.201  0.409  0.273  0.545 -0.2  0.267 0.0667 0.768
## 9 Clostrid~  0.00550 0.547  0.567  0.347  0.787 -0.133  0.933 0.8  0.871
## 10 Cronobac~ -0.105  0.929  0.511  0.303  0.719 -0.0667 0.533 0.467 1
## # i 102 more rows
## # i 2 more variables: Study <chr>, lab <chr>

```

fplot\_assoc





```

ggsave(
  filename = "Figure7.jpeg",
  plot = fplot_assoc,
  width = 7,
  height = 4.5,
  units = "in",
  dpi = 600,          # high-end journal quality
  bg = "white"
)

assoc_long |>
  mutate(flag = (p.adj <= 0.01) & (auc < 0.25 | auc > 0.75)) |>
  group_by(genus) |>
  summarise(any_keep = any(flag),
            m_auc = mean(auc),
            .groups = "drop") |>
  filter(any_keep) |>
  arrange(m_auc) |>
  pull(genus)

## [1] "Limivivens"

```

### 3 Merge for multivariate modelling and ML

Here we get into the distinction between simply doing pooled data analysis, and a more structured machine learning approach that is appropriate for microbiome meta-analysis.

Relative abundances were filtered for a minimum mean abundance of 0.001, and 'log.std' normalization was applied. Per the SIAMCAT documentation:

log-transforms features and applies z-score standardization. This method requires both a pseudocount (log.n0) and sd.min.q

```

merged_rel_abuns <- list(Borgo = borgo_RA,
                        Mack = mack_RA,
                        Proch = proch_RA,
                        Monte = monte_RA) |>
  # restrict to common subset, remove unresolved
  map(~ as.data.frame(.) |>
        rownames_to_column('genus') |>
        as_tibble() |>
        filter(genus %in% common_genera) |>
        column_to_rownames('genus')) |>
  bind_cols()

# needs same features
merged_feat <- phyloseq::otu_table(merged_rel_abuns,
                                   taxa_are_rows = TRUE)

# see helper function at top

```

```

merged_meta <- bind_rows(select_basemeta(borgo_meta_bmi_quant),
                          select_basemeta(mack_meta_full_bmi),
                          select_basemeta(proch_meta_bmi_quant),
                          select_basemeta(monte_meta_bmi_quant))[colnames(
                            merged_feat),]

merged_label <- create.label(meta=merged_meta,
                             label="group",
                             case="ANT1",
                             control="HC")

sc_merged <- siamcat(feats = merged_feat,
                    label = merged_label,
                    meta = merged_meta) |>
  filter.features(filter.method = "abundance", cutoff = 0.001) |>
  filter.features(filter.method = "prevalence",
                  cutoff = 0.01) |>
  # pseudocount + log + z-score
  normalize.features(norm.method = 'log.std',
                    norm.param=list(log.n0=1e-05,
                                    sd.min.q=0))

# print/visualize data
sc_merged

## siamcat-class object
## label()          Label object:          156 HC and 146 ANT1 samples
## filt_feats()     Filtered features:     357 features after prevalence filtering
## norm_feats()     Normalized features:    357 features normalized using log.std
##
## contains phyloseq-class experiment-level object @phyloseq:
## phyloseq@otu_table() OTU Table:          [ 359 taxa and 302 samples ]
## phyloseq@sam_data()  Sample Data:         [ 302 samples by 3 sample variables ]
merged_meta |> count(group) |> arrange(desc(n))

##   group   n
## 1    HC 156
## 2   ANT1 146

```

## 4 Building cross-validated LASSO models

(from the SIAMCAT vignette)

```

# create tibble to store all the predictions
set.seed(1)
auroc.all.S2S <- tibble(study.train=character(0),
                        study.test=character(0),
                        AUC=double(0))
cohorts <- unique(merged_meta$study)

merged_feats <- merged_rel_abuns

# and a list to save the trained SIAMCAT objects

```

```

sc.list.study2study <- list()
# replaced 'for (i in datasets) from vignette'
for (i in cohorts){
  # restrict to a single study
  meta.train <- merged_meta |>
    filter(study==i) |>
    as.data.frame()

  # create SIAMCAT object
  sc.obj.train <- siamcat(feat=merged_feat, label=create.label(
    meta=meta.train, label='group', case='ANT1', control='HC'),
    meta=meta.train)
  # normalize features
  sc.obj.train <- normalize.features(sc.obj.train,
                                     norm.method = 'log.std',
                                     norm.param=list(log.n0=1e-05,
                                                     sd.min.q=0),feature.type = 'original')
  # Create data split
  sc.obj.train <- create.data.split(sc.obj.train,
                                     num.folds = 10, num.resample = 10)
  # train LASSO model
  sc.obj.train <- train.model(sc.obj.train, method='lasso')

  ## apply trained models to other datasets

  # loop through datasets again
  for (i2 in cohorts){
    if (i == i2){
      # make and evaluate cross-validation predictions (same dataset)
      sc.obj.train <- make.predictions(sc.obj.train)
      sc.obj.train <- evaluate.predictions(sc.obj.train)
      auroc.all.S2S <- auroc.all.S2S |>
        add_row(study.train=i, study.test=i,
                AUC=eval_data(sc.obj.train)$auroc |> as.double())
    } else {
      # make and evaluate on the external datasets
      # use meta.ind here, since we want only one sample per subject!
      meta.test <- merged_meta |>
        filter(study==i2) |>
        as.data.frame()
      sc.obj.test <- siamcat(feat=merged_feat,
                            label=create.label(meta=meta.test,
                                                label='group',
                                                case='ANT1', control='HC'),
                            meta=meta.test)
      # make holdout predictions
      sc.obj.test <- make.predictions(sc.obj.train,
                                      siamcat.holdout = sc.obj.test)
      sc.obj.test <- evaluate.predictions(sc.obj.test)
      auroc.all.S2S <- auroc.all.S2S |>
        add_row(study.train=i, study.test=i2,
                AUC=eval_data(sc.obj.test)$auroc |> as.double())
    }
  }
}

```

```

}
# save the trained model
sc.list.study2study[[i]] <- sc.obj.train
}

test.average <- auroc.all.S2S |>
  filter(study.train!=study.test) |>
  group_by(study.test) |>
  summarise(AUC=mean(AUC), .groups='drop') |>
  mutate(study.train="Average")

train.average <- auroc.all.S2S |>
  filter(study.train != study.test) |>
  group_by(study.train) |>
  summarise(AUC = mean(AUC), .groups='drop') |>
  mutate(study.test = "Average")

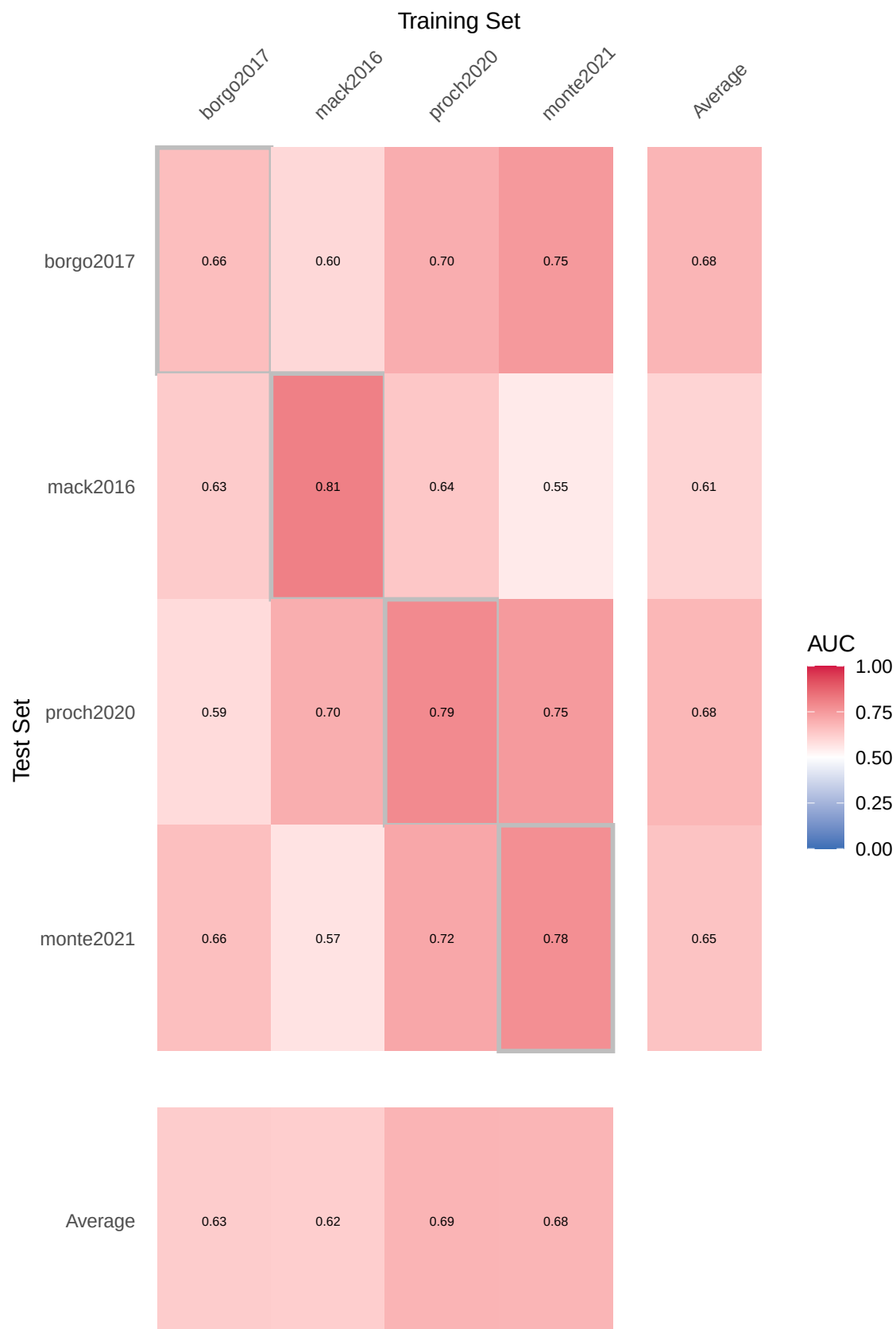
# combine AUROC values with test average
bind_rows(auroc.all.S2S, test.average, train.average) |>
  # highlight cross validation versus transfer results
  mutate(CV=study.train == study.test) |>
  # for facetting later
  mutate(split=case_when(study.train=='Average'~'Average',
                          TRUE~'none')) |>
  mutate(split=factor(split, levels = c('none', 'Average')) |>

mutate(split2=case_when(study.test=='Average'~'Average',
                        TRUE~'none')) |>
mutate(split2=factor(split2, levels = c('none', 'Average')) |>

# convert to factor to enforce ordering
mutate(study.train=factor(study.train,
                          levels=c(levels(cohorts), 'Average')) |>
mutate(study.test=factor(study.test,
                          levels=c(rev(levels(cohorts)), 'Average')) |>
ggplot(aes(y=study.test, x=study.train, fill=AUC, size=CV, color=CV)) +
  geom_tile() + theme_minimal() +
  # text in tiles
  geom_text(aes_string(label="format(AUC, digits=2)"),
            col='black', size=2)+
  # color scheme
  scale_fill_gradient2(low = "#3B6FB6",
                       mid = "white",
                       high = "#D41645",
                       midpoint = 0.5, limits = c(0, 1)) +
  # axis position/remove boxes/ticks/facet background/etc.
  scale_x_discrete(position='top') +
  theme(axis.line=element_blank(),
        axis.ticks = element_blank(),
        axis.text.x.top = element_text(angle=45, hjust=.1),
        panel.grid=element_blank(),
        panel.border=element_blank(),
        strip.background = element_blank(),
        strip.text = element_blank()) +

```

```
xlab('Training Set') + ylab('Test Set') +  
scale_color_manual(values=c('#FFFFFF00', 'grey'), guide=FALSE) +  
scale_size_manual(values=c(0, 1), guide=FALSE) +  
facet_grid(split2~split, scales = 'free', space = 'free')
```



```

ggsave("figures/S2SHeatmap_with_averages.png", (bind_rows(auroc.all.S2S, test.average, train.average) |>
  # highlight cross validation versus transfer results
  mutate(CV=study.train == study.test) |>
  # for facetting later
  mutate(split=case_when(study.train=='Average'~'Average',
                          TRUE~'none')) |>
  mutate(split=factor(split, levels = c('none', 'Average')))) |>

mutate(split2=case_when(study.test=='Average'~'Average',
                        TRUE~'none')) |>
mutate(split2=factor(split2, levels = c('none', 'Average')))) |>

# convert to factor to enforce ordering
mutate(study.train=factor(study.train,
                          levels=c(levels(cohorts), 'Average')))) |>
mutate(study.test=factor(study.test,
                          levels=c(rev(levels(cohorts)), 'Average')))) |>
ggplot(aes(y=study.test, x=study.train, fill=AUC, size=CV, color=CV)) +
  geom_tile() + theme_minimal() +
  # text in tiles
  geom_text(aes_string(label="format(AUC, digits=2)"),
            col='black', size=2)+
  # color scheme
  scale_fill_gradient2(low = "#3B6FB6",
                       mid = "white",
                       high = "#D41645",
                       midpoint = 0.5, limits = c(0, 1)) +
  # axis position/remove boxes/ticks/facet background/etc.
  scale_x_discrete(position='top') +
  theme(axis.line=element_blank(),
        axis.ticks = element_blank(),
        axis.text.x.top = element_text(angle=45, hjust=.1),
        panel.grid=element_blank(),
        panel.border=element_blank(),
        strip.background = element_blank(),
        strip.text = element_blank()) +
  xlab('Training Set') + ylab('Test Set') +
  scale_color_manual(values=c('#FFFFFF00', 'grey'), guide=FALSE) +
  scale_size_manual(values=c(0, 1), guide=FALSE) +
  facet_grid(split2~split, scales = 'free', space = 'free'),
            width = 2000, height = 1000, units = "px", bg = "white")

auroc_tbl_truloso <- tibble(train = character(),
                             test = character(),
                             AUC = double())
model_store_truloso <- list()
cohorts_truloso <- unique(merged_meta$study)
set.seed(1)
for (te in cohorts_truloso) {
  # Define training (all except current test cohort)
  train_idx_truloso <- which(merged_meta$study != te)

```

```

test_idx_truloso <- which(merged_meta$study == te)

# SIAMCAT on training data
sc_tr_truloso <- siamcat(feat = as.data.frame(merged_rel_abuns)[,train_idx_truloso, drop = FALSE],
  label = create.label(meta = merged_meta[train_idx_truloso, ],
    label = "group",
    case = "ANT1",
    control = "HC"),
  meta = merged_meta[train_idx_truloso, ]) |>
  filter.features(filter.method = "pass") |>
  normalize.features(norm.method = "log.std",
    norm.param = list(log.n0 = 1e-05, sd.min.q = 0)) |>
  create.data.split(num.folds = 10, num.resample = 10) |>
  train.model(method = "lasso", perform.fs = TRUE,
    param.fs = list(no_features = 150,
      method = "AUC",
      direction = "absolute"))

model_store_truloso[[te]] <- sc_tr_truloso

# Apply trained model to test set (held-out study)
sc_te_truloso <- siamcat(feat = as.data.frame(merged_rel_abuns)[, test_idx_truloso, drop = FALSE],
  label = create.label(meta = merged_meta[test_idx_truloso, ],
    label = "group",
    case = "ANT1", control = "HC"),
  meta = merged_meta[test_idx_truloso, ])

# Predict and evaluate
sc_te_truloso <- make.predictions(sc_tr_truloso, siamcat.holdout = sc_te_truloso) |>
  evaluate.predictions()

auroc_tbl_truloso <- add_row(auroc_tbl_truloso,
  train = paste(setdiff(cohorts_truloso, te), collapse = "+"),
  test = te,
  AUC = as.numeric(sc_te_truloso@eval_data$auroc))
}

#Combine into one neat chart:
# Add identifier for LOSO
auroc.all.S2S$Type <- "Study-to-Study"
auroc_tbl_truloso$Type <- "LOSO"

# For LOSO we don't really care about "train" as it's "all except test",
# so let's give it a cleaner label
auroc_tbl_truloso <- auroc_tbl_truloso %>%
  rename(study.test = test,
    study.train_original = train) %>%
  mutate(study.train = "LOSO") %>% # label training scheme
  dplyr::select(study.train, study.test, AUC, Type) %>%
  filter(!is.na(study.test))

# Get unique study names
study_names <- unique(auroc.all.S2S$study.train)

```



```

# Combine both tables
auroc_tbl_combined <- bind_rows(auroc.all.S2S, auroc_tbl_truloso)

library(RColorBrewer)

# Assign colors: studies from Set1, LOSO = orange

study_colors <- setNames(
  c(brewer.pal(min(length(study_names), 8), "Set1"), "orange"),
  c(study_names, "LOSO")
)

# --- Ensure LOSO plotted last for each test study ---
auroc_tbl_combined <- auroc_tbl_combined %>%
  mutate(
    study.train = factor(study.train, levels = c(study_names, "LOSO")),
    study.test = factor(study.test, levels = unique(auroc.all.S2S$study.test))
  )

# --- Plot ---

# Mean AUROC per test study
avg_tbl <- auroc_tbl_combined %>%
  filter(Type == "Study-to-Study") %>%
  group_by(study.test) %>%
  summarise(mean_AUC = mean(AUC, na.rm = TRUE), .groups = "drop") %>%
  mutate(xnum = seq_along(study.test))

fplot_combined_average <- ggplot(
  auroc_tbl_combined,
  aes(x = study.test, y = AUC, fill = study.train)
) +
  geom_col(position = position_dodge(width = 0.7), width = 0.6) +
  geom_text(
    aes(label = sprintf("%.2f", AUC)),
    position = position_dodge(width = 0.7),
    vjust = -0.5, size = 2
  ) +
  #
  # --- mean line ---
  geom_segment(
    data = avg_tbl,
    aes(
      x = xnum - 0.38,
      xend = xnum + 0.38,
      y = mean_AUC,
      yend = mean_AUC,
      linetype = "Mean AUROC"
    ),
    inherit.aes = FALSE,
    linewidth = 1,
    color = "black",
    alpha = 0.5
  ) +

```

```

# --- unified legend ---
scale_fill_manual(
  name = "Training / Summary",
  values = study_colors
) +

scale_linetype_manual(
  name = "Training / Summary",
  values = c("Mean AUROC" = "dashed")
) +

labs(
  x = "Test Study",
  y = "AUROC",
  title = "Study-to-Study vs LOSO AUROC"
) +

coord_cartesian(ylim = c(0, 1)) +
theme_minimal(base_size = 13) +
theme(
  legend.position = "bottom",
  axis.text.x = element_text(angle = 45, hjust = 1)
)

ggsave(
  "figures/auroc_combined_plot_average.pdf",
  plot = fplot_combined_average,
  width = 12, height = 7, units = "in",
  device = cairo_pdf, bg = "white"
)

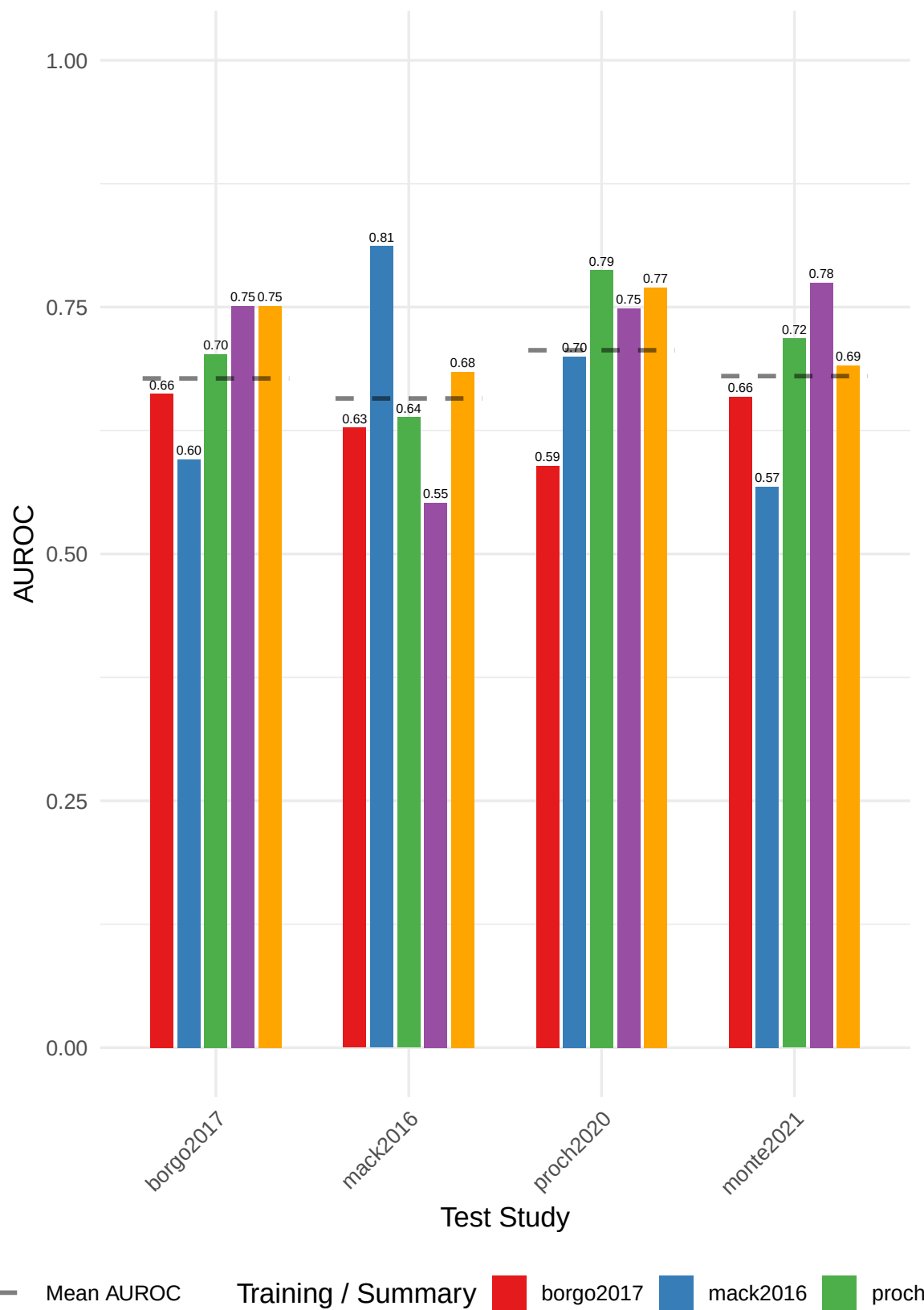
ggsave(
  "figures/auroc_combined_plot_average.jpg",
  plot = fplot_combined_average,
  width = 12, height = 7, units = "in",
  dpi = 600, bg = "white"
)

#
fplot_combined <- ggplot(auroc_tbl_combined,
  aes(x = study.test, y = AUC, fill = study.train)) +
  geom_col(position = position_dodge(width = 0.7), width = 0.6) +
  geom_text(aes(label = sprintf("%.2f", AUC)),
    position = position_dodge(width = 0.7),
    vjust = -0.5, size = 2) +
  scale_fill_manual(values = study_colors) +
  labs(
    x = "Test Study",
    y = "AUROC",
    fill = "Training Study",
    title = "Study-to-Study vs LOSO AUROC",
    caption = "Each test study shows 4 single-study transfers + 1 LOSO (black)"
  ) +
  ylim(0, 1) +

```

```
theme_minimal(base_size = 13) +  
theme(  
  legend.position = "bottom",  
  axis.text.x = element_text(angle = 45, hjust = 1)  
)  
  
# Print the plot  
fplot_combined_average
```

## Study-to-Study vs LOSO AUROC



```
ggsave("figures/auroc_combined_plot.png", fplot_combined,
       width = 8, height = 5, dpi = 300, bg = "white")
```

## 5 LOSO coefficients for diagnostics (raw values)

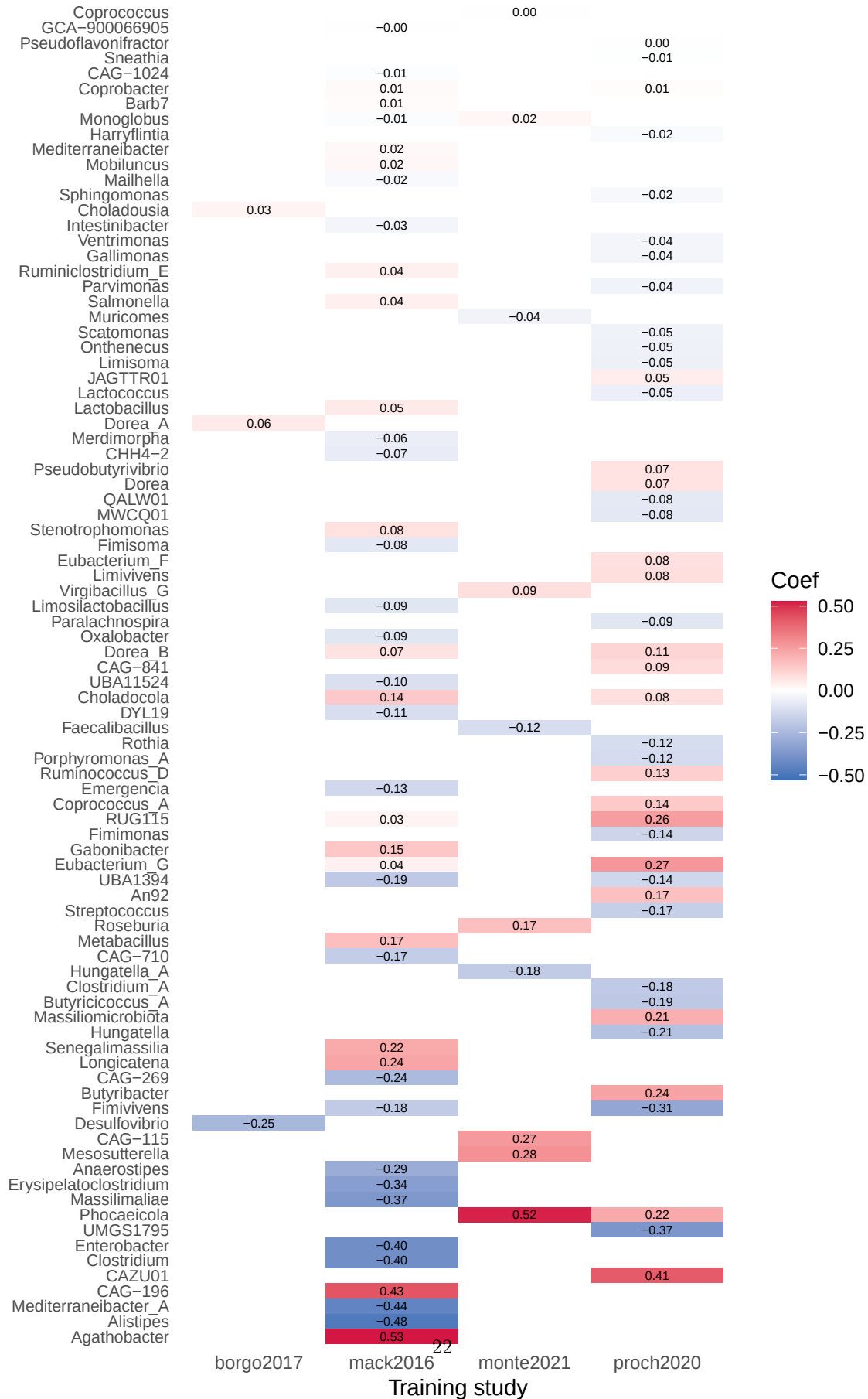
```
#-- Each LOSO model keeps only genera with non-zero weights -----
# goal: one point per (train-study × genus) so we can see selection pattern
```

```
coef_df <- map2_df(sc.list.study2study, names(sc.list.study2study),
                  \(sc, cohort){
  wm <- weight_matrix(sc) |> as.matrix()
  keep <- wm != 0
  tibble(genus = rownames(wm)[keep],
         coef = wm[keep],
         train = cohort) }) |>
  filter(!is.na(genus))
```

```
# order genera by mean |coef| across studies
feat_ord <- coef_df |>
  group_by(genus) |>
  summarise(avg = mean(abs(coef)), .groups = "drop") |>
  arrange(desc(avg)) |>
  pull(genus)
coef_df$genus <- factor(coef_df$genus, levels = feat_ord)
```

```
ggplot(coef_df, aes(train, genus, fill = coef)) +
  geom_tile() +
  geom_text(aes(label = sprintf("%.2f", coef)), size = 2) +
  scale_fill_gradient2(low = "#3B6FB6",
                      mid = "white",
                      high = "#D41645",
                      limits = c(-max(abs(coef_df$coef)),
                                max(abs(coef_df$coef))),
                      name = "Coef") +
  theme_minimal() +
  theme(axis.text.y = element_text(size = 8),
        panel.grid = element_blank()) +
  labs(x = "Training study", y = NULL,
       title = "Raw nonzero coefficients per LOSO model",
       caption = "Tiles are signed weights; blank tiles mean the genus was not selected in that model.")
```

## Raw nonzero coefficients per LOSO model



Tiles are signed weights; blank tiles mean the genus was not selected in that model.

```

# plot figure
ggsave("Figure11.png", ggplot(coef_df, aes(train, genus, fill = coef)) +
  geom_tile() +
  geom_text(aes(label = sprintf("%.2f", coef)), size = 2) +
  scale_fill_gradient2(low = "#3B6FB6",
    mid = "white",
    high = "#D41645",
    limits = c(-max(abs(coef_df$coef)),
      max(abs(coef_df$coef))),
    name = "Coef") +
  theme_minimal() +
  theme(axis.text.y = element_text(size = 8),
    panel.grid = element_blank()) +
  labs(x = "Training study", y = NULL,
    title = "Raw nonzero coefficients per LOSO model",
    caption = "Tiles are signed weights; blank tiles mean the genus was not selected in that model.",
    width = 15, height = 10, units = "in", dpi = 300, bg = "white")

```

#LOSO normalized feature weights

```

weights_df <- map2_df(sc.list.study2study,
  names(sc.list.study2study), function(sc, cohort){
    feature_weights(sc) |>
    as.data.frame() |>
    rownames_to_column("genus") |>
    as_tibble() |>
    select(genus, median.rel.weight, mean.rel.weight, percentage) |>
    mutate(Study = cohort,
      r.med = rank(-abs(median.rel.weight)))
  })

norm_tbl <- weights_df |>
  group_by(Study) |>
  summarise(sum_med = sum(abs(median.rel.weight)), .groups = "drop")

plot_df <- weights_df |>
  inner_join(norm_tbl, by = "Study") |>
  mutate(norm.weight = median.rel.weight / sum_med) |>
  filter(genus %in% feat_ord) |>
  mutate(genus = factor(genus, levels = rev(feat_ord)),
    Study = factor(Study, levels = cohorts),
    rank_show = if_else(r.med <= 20, r.med, NA_real_))

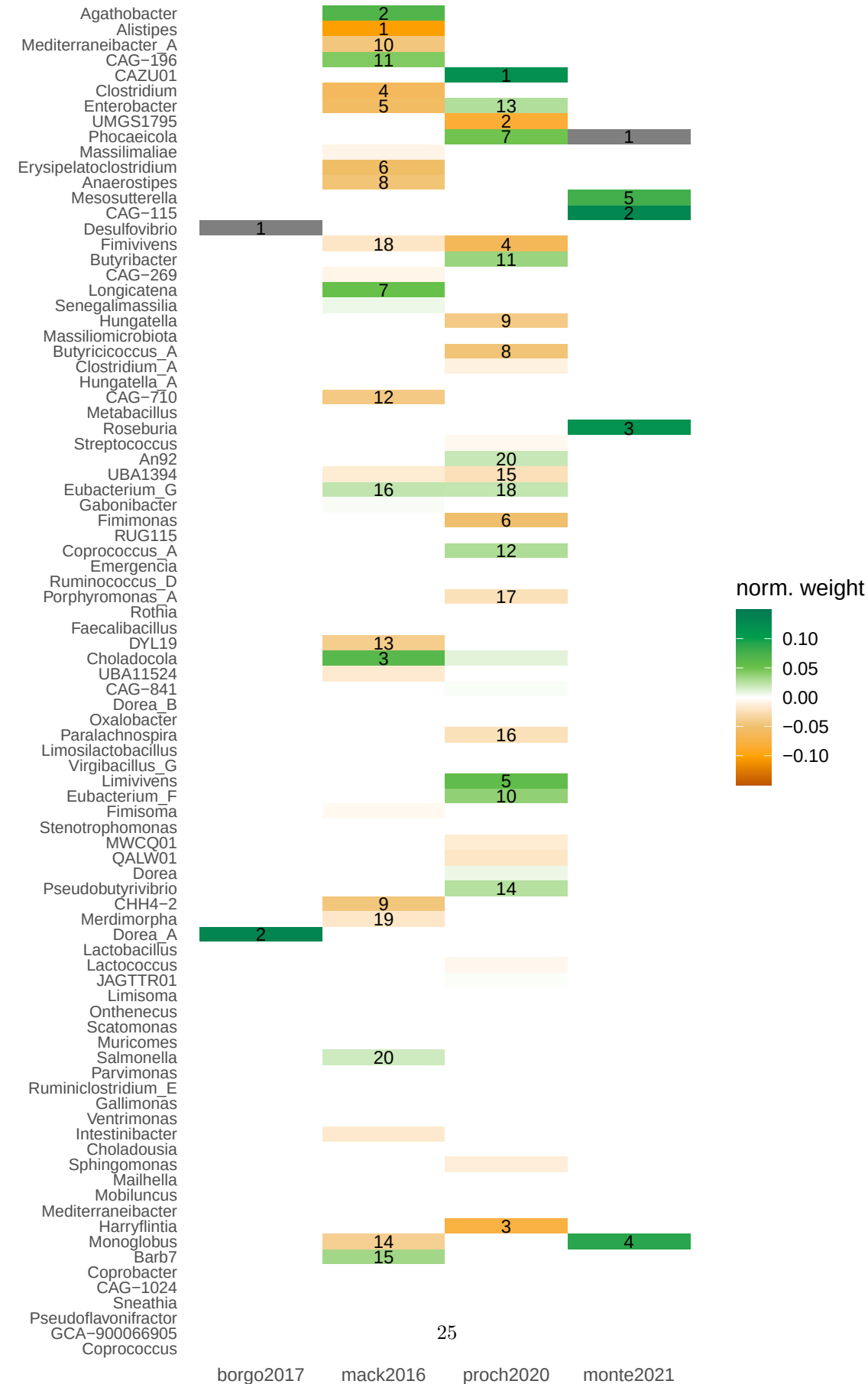
ggplot(plot_df, aes(Study, genus, fill = norm.weight)) +
  geom_tile() +
  geom_text(aes(label = rank_show), size = 3) +
  scale_fill_gradientn(colours = rev(c("#007A53", "#009F4D", "#6CC24A", "white",
    "#EFC06E", "#FFA300", "#BE5400")),
    limits = c(-0.15, 0.15), name = "norm. weight") +
  theme_minimal() +
  theme(axis.text.y = element_text(size = 8),
    panel.grid = element_blank()) +
  labs(x = NULL, y = NULL,
    title = "Normalized median relative weights (top-ranked genera)",

```

```
caption='Colors are relative importance after model-size normalization; numbers = within-model
```



## Normalized median relative weights (top-ranked genera)



tive importance after model-size normalization; numbers = within-model rank (1–20).

```

ggsave("Figure 12.png", ggplot(plot_df, aes(Study, genus, fill = norm.weight)) +
  geom_tile() +
  geom_text(aes(label = rank_show), size = 3) +
  scale_fill_gradientn(colours = rev(c('#007A53', '#009F4D', '#6CC24A', 'white',
                                         '#EFC06E', '#FFA300', '#BE5400')),
                      limits = c(-0.15, 0.15), name = "norm. weight") +
  theme_minimal() +
  theme(panel.background = element_rect(fill = "white", color = NA),
        plot.background = element_rect(fill = "white", color = NA),
        legend.background = element_rect(fill = "white", color = NA),
        legend.box.background = element_rect(fill = "white", color = NA),
        axis.text.y = element_text(size = 8),
        panel.grid = element_blank()) +
  labs(x = NULL, y = NULL,
        caption='Colors are relative importance after model-size normalization; numbers = within-model
        width = 15, height = 10, units = "in", dpi = 300, bg = "white"))

stability_tbl <- coef_df %>%
  group_by(genus) %>%
  summarise(
    n_models = n_distinct(train),
    n_selected = n(),
    share_selected = n_selected / n_models,
    share_pos = mean(coef > 0),
    share_neg = mean(coef < 0),
    sign_consistency = pmax(share_pos, share_neg),
    mean_abs_coef = mean(abs(coef)),
    .groups = "drop"
  ) %>%
  mutate(stable = share_selected >= 0.5 & sign_consistency >= 0.8)

```

## 6 Meta-Analysis with Linear Mixed-Effect Models

```

sc_merged_assoc <- siamcat(feats = merged_feats,
                          label = merged_label,
                          meta = merged_meta)
assoc_lmer <- sc_merged_assoc |>
  filter.features(filter.method = "abundance", cutoff = 0.001) |>
  filter.features(filter.method = "prevalence", cutoff = 0.01) |>
  # pseudocount + log + z-score
  normalize.features(norm.method = 'log.std',
                    norm.param=list(log.n0=1e-05,
                                    sd.min.q=0)) |>
  check.associations(formula = 'feat~label+(1|study)', test='lm')

meta_assoc <- assoc_lmer@associations$assoc.results |>
  rownames_to_column('genus') |>
  as_tibble() |>
  add_column(Study='Meta-Analysis') |>
  bind_rows(assoc_long) # single study estimates from above

```

```

# --- FIX 1: match merged_meta study ids -> pretty names used in meta_assoc ---
study_name_map <- c(
  "borgo2017" = "Borgo2017",
  "mack2016" = "Mack2016",
  "proch2020" = "Proch2020",
  "monte2021" = "Monte2021"
)

merged_meta_pretty <- merged_meta |>
  mutate(study_name = dplyr::recode(as.character(study), !!!study_name_map))

# Force metric consistency: pooled uses fc too
meta_metric <- "fc"
x_axis_label <- "Linear model effect size [95% CI]"

# If meta_assoc lacks fc CIs, derive a quick Wald CI from estimate & two-sided p
# (uses unadjusted p.val; OK for display - BH-adjust stays in p.val_adj)
if (!all(c("fc.ci.l", "fc.ci.h") %in% names(meta_assoc))) {
  z_fun <- function(p) { p <- pmax(pmin(p, 1 - 1e-12), 1e-12); qnorm(1 - p/2) }
  meta_assoc <- meta_assoc |>
    mutate(
      .z = z_fun(p.val),
      .se = ifelse(.z > 0, abs(fc)/.z, NA_real_),
      fc.ci.l = fc - 1.96*.se,
      fc.ci.h = fc + 1.96*.se
    )
}

# Tell the adapter where CIs live (so your existing code picks them up)
fc_ci_l <- "fc.ci.l"
fc_ci_h <- "fc.ci.h"
has_fc_ci <- TRUE
has_auc_ci <- FALSE

# ----- knobs -----
p_thresh <- 0.05 # pooled meta q-value cutoff
p_thresh_individual <- 0.1 # per-study q-value cutoff

# ----- pick CI source from meta_assoc -----
has_fc_ci <- all(c("fc.ci.l", "fc.ci.h") %in% names(meta_assoc)) ||
  all(c("fc_ci_l", "fc_ci_h") %in% names(meta_assoc)) ||
  all(c("ci.l", "ci.h") %in% names(meta_assoc))

has_auc_ci <- all(c("auc.ci.l", "auc.ci.h") %in% names(meta_assoc))

# helper to return the first available column name from a candidate set
col_if_any <- function(cands) {
  nm <- cands[cands %in% names(meta_assoc)]
  if (length(nm) == 0) NULL else nm
}

fc_ci_l <- col_if_any(c("fc.ci.l", "fc_ci_l", "ci.l"))
fc_ci_h <- col_if_any(c("fc.ci.h", "fc_ci_h", "ci.h"))

```

```

# choose metric/CI consistently for pooled rows
if (has_fc_ci) {
  meta_metric <- "fc"          # plot on effect size
  meta_lower_col <- fc_ci_l
  meta_upper_col <- fc_ci_h
  x_axis_label <- "Linear model effect size [95% CI]"
} else if (has_auc_ci) {
  meta_metric <- "auc"          # fall back to AUC if fc CI missing
  meta_lower_col <- "auc.ci.l"
  meta_upper_col <- "auc.ci.h"
  x_axis_label <- "AUC [95% CI]"
} else {
  stop("meta_assoc has no recognizable 95% CI columns (looked for fc.ci.* / auc.ci.*).")
}

# ----- pooled meta rows (for big square + CIs) -----
test_res_all_studies_df <- meta_assoc |>
  filter(grepl("^Meta", Study)) |>
  transmute(
    feat1 = "disease_meta",
    feat1_group = "Meta-analysis",
    feat2 = genus,
    effect_size = .data[[meta_metric]],
    lower95CI = .data[[meta_lower_col]],
    upper95CI = .data[[meta_upper_col]],
    p.val = p.val,
    p.val_adj = p.adj
  )

# ----- per-study rows (for small points) -----
test_res_individual_studies_df <- meta_assoc |>
  filter(!grepl("^Meta", Study)) |>      # the four cohorts
  transmute(
    feat1 = paste0("disease_", Study),
    feat1_group = Study,                  # pretty study name
    feat2 = genus,
    effect_size = fc,                     # per-study effect size
    p.val = p.val,
    p.val_adj = p.adj
  )

# ----- selection & ordering driven by pooled q-values -----
genus_sel_vec <- test_res_all_studies_df |>
  filter(p.val_adj < p_thresh) |>
  pull(feat2) |> unique()

bac_levels <- test_res_all_studies_df |>
  filter(feat2 %in% genus_sel_vec) |>
  arrange(desc(effect_size)) |>
  pull(feat2)

# ----- counts for legend (N ANT1; N Healthy) -----
# FIX: use merged_meta_pretty (so names match) instead of merged_meta

```

```

study_counts <- merged_meta_pretty |>
count(study_name, group) |>
tidyr::pivot_wider(names_from = group, values_from = n, values_fill = 0) |>
transmute(
  study_name,          # "Borgo", "Mack", "Proch", "Monte"
  N_Group1 = HC,       # legend format expects (ANT1; HC) later
  N_Group2 = ANT1
)

# ----- plotting dfs (ANT1/Healthy wording) -----
plot_df <- test_res_individual_studies_df |>
filter(feet2 %in% genus_sel_vec) |>
mutate(
  bac = factor(feet2, levels = rev(bac_levels)),
  study_name = feat1_group,
  enriched_in = case_when(
    effect_size > 0 & p.val_adj < p_thresh_individual ~ "ANT1",
    effect_size < 0 & p.val_adj < p_thresh_individual ~ "Healthy",
    TRUE ~ "n.s."
  ),
  enriched_in = factor(enriched_in, levels = c("ANT1", "Healthy", "n.s."))
) |>
left_join(study_counts, by = "study_name") |>      # attach (N ANT1; N HC)
mutate(
  N_total = N_Group1 + N_Group2,
  study_name = factor(study_name, levels = c("Meta-analysis", sort(unique(study_name))))
)

# build legend labels
study_lab_df <- plot_df |>
select(study_name, N_Group1, N_Group2) |>
distinct() |>
mutate(study_lab = paste0(study_name, " (", N_Group2, ";;", N_Group1, ")"))

# add pooled totals to the legend
study_lab_df <- bind_rows(
  study_lab_df,
  tibble(
    study_name = "Meta-analysis",
    N_Group1 = sum(study_lab_df$N_Group1, na.rm = TRUE),
    N_Group2 = sum(study_lab_df$N_Group2, na.rm = TRUE),
    study_lab = paste0("Meta-analysis (",
      sum(study_lab_df$N_Group2, na.rm = TRUE), ";;",
      sum(study_lab_df$N_Group1, na.rm = TRUE), ")")
  )
)

# subset of pooled rows for plotted genera; compute fill label
plot_df_meta <- test_res_all_studies_df |>
filter(feet2 %in% bac_levels) |>
mutate(
  bac = factor(feet2, levels = rev(bac_levels)),
  study_name = feat1_group,

```

```

    enriched_in = factor(ifelse(effect_size > 0, "ANT1", "Healthy"),
                          levels = c("ANT1", "Healthy"))
  )

# ----- shapes/palette (compact defaults; override if you have your own) -----
study_shapes <- c(Borgo2017 = 0, Mack2016 = 1, Proch2020 = 2, Monte2021 = 3, "Meta-analysis" = 22)
if (!exists("all_colors")) all_colors <- c(ANT1 = "#D41645", Healthy = "#3B6FB6")

# force both categories to appear in the fill legend (even if one absent)
fill_limits <- c("ANT1", "Healthy")

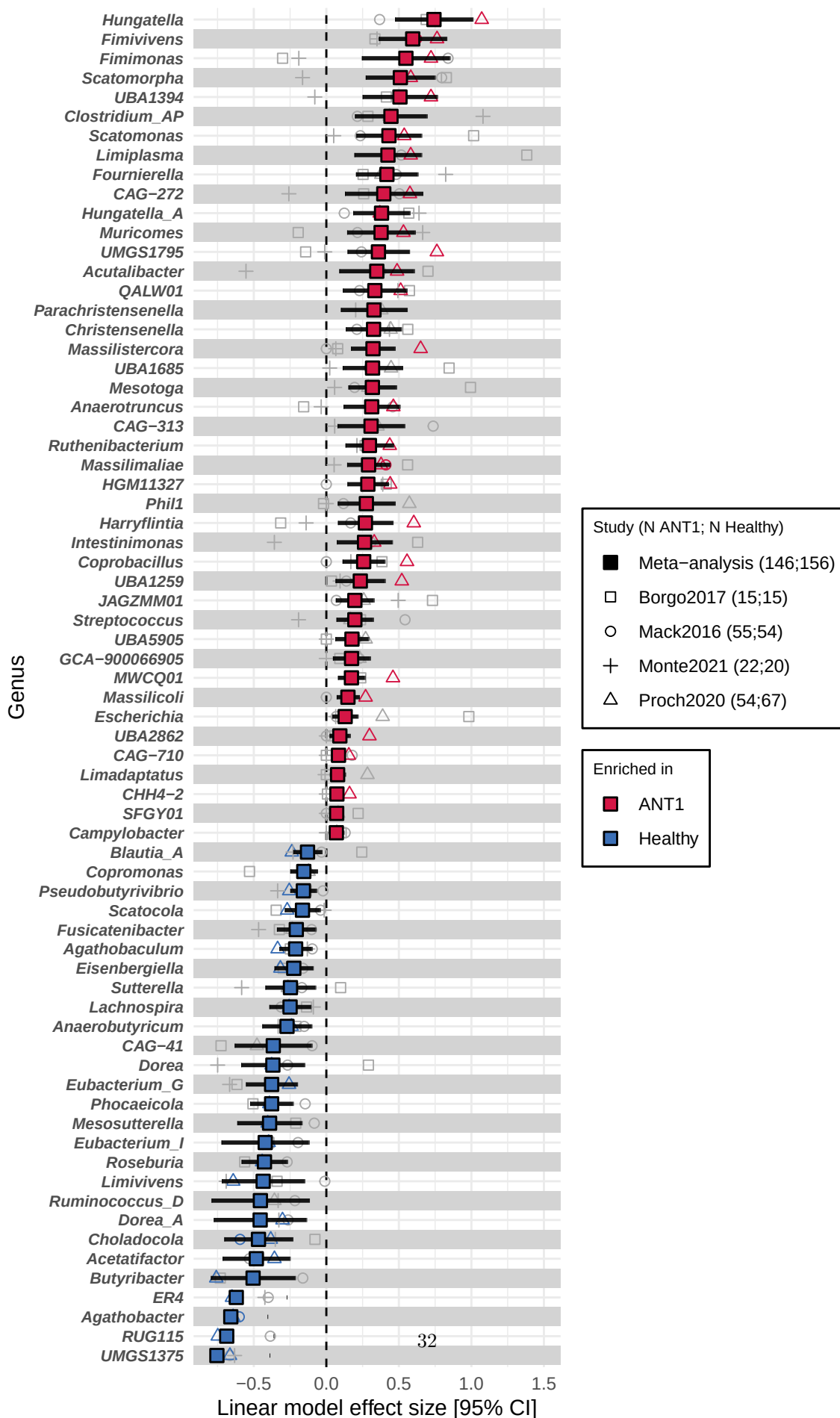
# ----- axis label you can pass to labs(x = x_axis_label) -----
# x_axis_label

pt_forest <- plot_df |>
  ggplot(aes(x = effect_size, y = bac)) +
  ggforestplot::geom_stripes(odd = "lightgrey", even = "#00000000") +
  geom_vline(xintercept = 0, linetype = "dashed", color = "black") +
  # per-study points
  geom_point(aes(shape = study_name, color = enriched_in), size = 2, alpha = 1) +
  # pooled meta CI + point
  geom_errorbar(data = plot_df_meta,
               aes(xmin = lower95CI, xmax = upper95CI),
               width = 0, size = 1, color = "black",
               alpha = 0.9, show.legend = FALSE, na.rm = TRUE) +
  geom_point(data = plot_df_meta,
             aes(fill = enriched_in, shape = "Meta-analysis"),
             size = 3, stroke = 0.75) +
  scale_shape_manual(
    values = study_shapes[match(levels(plot_df$study_name),
                                names(study_shapes))],
    labels = study_lab_df$study_lab[match(levels(plot_df$study_name),
                                           study_lab_df$study_name)],
    drop = FALSE
  ) +
  (if (exists("theme_paper")) theme_paper else theme_minimal()) +
  # FIX: keep both fill categories visible even if one is absent
  scale_fill_manual(values = all_colors, limits = fill_limits, drop = FALSE) +
  scale_color_manual(values = c(all_colors, "n.s." = "darkgrey"), guide = "none") +
  labs(
    x = x_axis_label,
    y = "Genus",
    shape = "Study (N ANT1; N Healthy)",
    fill = "Enriched in"
  ) +
  theme(
    axis.text.y = element_text(face = "bold.italic", size = 8),
    legend.background = element_rect(fill = "white", color = "black"),
    legend.title = element_text(size = 8),
    legend.position = "right"
  ) +
  guides(
    shape = guide_legend(

```

```
    override.aes = list(fill = "black", colour = "black")
  ),
  fill = guide_legend(
    reverse = FALSE,
    override.aes = list(shape = 22)
  )
) +
xlim(c(-0.8, 1.5))
```

pt\_forest





## 7 Descriptive Analysis

```
# --- Merge datasets with Study labels ---

otu_borgo <- otu_table(as.matrix(borgo_rarefied), taxa_are_rows = TRUE)
otu_mack  <- otu_table(as.matrix(mack_rarefied), taxa_are_rows = TRUE)
otu_proch <- otu_table(as.matrix(proch_rarefied), taxa_are_rows = TRUE)
otu_monte <- otu_table(as.matrix(monte_rarefied), taxa_are_rows = TRUE)

# 2. Create sample metadata
# Sample IDs from OTU table
borgo_samples <- colnames(borgo_rarefied)

# Subset metadata rows where rowname is in proch_samples
borgo_meta_sub <- borgo_meta[rownames(borgo_meta) %in% borgo_samples, , drop = FALSE]

# Reorder metadata to match sample order in OTU table
borgo_meta_sub <- borgo_meta_sub[match(borgo_samples, rownames(borgo_meta_sub)), ]

# Now build sample_data
sample_borgo <- sample_data(data.frame(
  SampleID = borgo_samples,
  Study    = "borgo",
  Category = borgo_meta_sub$group,
  row.names = borgo_samples
))
sample_mack  <- sample_data(data.frame(SampleID = colnames(mack_rarefied),
                                       Study    = "mack",
                                       Category = mack_meta$group,
                                       row.names = colnames(mack_rarefied)))

# Sample IDs from your OTU table
proch_samples <- colnames(proch_rarefied_RA)

# Subset metadata rows where rowname is in proch_samples
proch_meta_sub <- proch_meta[rownames(proch_meta) %in% proch_samples, , drop = FALSE]

# Reorder metadata to match sample order in OTU table
proch_meta_sub <- proch_meta_sub[match(proch_samples, rownames(proch_meta_sub)), ]

# Now build sample_data
sample_proch <- sample_data(data.frame(
  SampleID = proch_samples,
  Study    = "proch",
  Category = proch_meta_sub$group,
  row.names = proch_samples
))
sample_monte <- sample_data(data.frame(SampleID = colnames(monte_rarefied),
                                       Study = rep("monte", 34),
                                       Category = monte_meta$group[rownames(monte_meta) %in% colnames(monte_rarefied)],
                                       row.names = colnames(monte_rarefied)))

ps_borgo <- phyloseq(otu_borgo, sample_borgo)
ps_mack  <- phyloseq(otu_mack,  sample_mack)
```

```

ps_proch <- phyloseq(otu_proch, sample_proch)
ps_monte <- phyloseq(otu_monte, sample_monte)

# 5. Merge into one big ps object
ps_combined <- merge_phyloseq(ps_borgo, ps_mack, ps_proch, ps_monte)

# --- Alpha Diversity ---
alpha_div <- estimate_richness(ps_combined, measures = c("Shannon", "Simpson", "Observed", "Chao1"))

alpha_div$SampleID <- rownames(alpha_div)
alpha_div <- left_join(alpha_div,
  data.frame(SampleID = sample_names(ps_combined),
    Study = sample_data(ps_combined)$Study,
    Category = sample_data(ps_combined)$Category),
  by = "SampleID")
#coerce NAs to Monte data
alpha_div$Study[is.na(alpha_div$Study)] <- "monte"
alpha_div$Category[is.na(alpha_div$Category)] <- sample_data(ps_combined)[sample_data(ps_combined)$Study
alpha_div$Category <- as.factor(alpha_div$Category)

# Example plot: Shannon diversity
p_alpha <- ggplot(alpha_div, aes(x = Study, y = Shannon, fill = Study)) +
  geom_boxplot(width = 0.6, alpha = 0.8) +
  geom_jitter(width = 0.2, size = 1.5, alpha = 0.7) +
  labs(title = "Alpha Diversity (Shannon)", y = "Shannon Index") +
  theme_minimal()

# recode study in alpha div:
alpha_div <- alpha_div %>% mutate(
  Study = recode(Study,
    "borgo" = "borgo2017",
    "mack" = "mack2016",
    "proch" = "proch2020",
    "monte" = "monte2022"))
comparisons <- list(
  c("borgo2017", "monte2022"),
  c("borgo2017", "proch2020"),
  c("borgo2017", "mack2016"),
  c("monte2022", "proch2020"),
  c("monte2022", "mack2016"),
  c("proch2020", "mack2016")
)

# Plot with comparisons
library("ggpubr")

p_alpha_shannon <- ggplot(alpha_div, aes(x = Study, y = Shannon, fill = Study)) +
  geom_boxplot(width = 0.6, alpha = 0.8) +
  geom_jitter(width = 0.2, size = 1.5, alpha = 0.7) +
  #labs(title = "Alpha Diversity (Shannon)",
  labs(y = "Shannon Index") +

```

```

theme_minimal() +
stat_compare_means(comparisons = comparisons,
                    method = "wilcox.test", # nonparametric test (common for alpha diversity)
                    label = "p.format")

ggsave(
  filename = "shannon_alpha_div.pdf",
  plot = p_alpha_shannon,
  width = 7, # typical 2-column width
  height = 4.5,
  units = "in",
  device = cairo_pdf
)

ggsave(
  filename = "shannon_alpha_div.jpg",
  plot = p_alpha_shannon,
  width = 7,
  height = 4.5,
  units = "in",
  dpi = 600,
  bg = "white"
)

p_alpha_simpson <- ggplot(alpha_div, aes(x = Study, y = Simpson, fill = Study)) +
  geom_boxplot(width = 0.6, alpha = 0.8) +
  geom_jitter(width = 0.2, size = 1.5, alpha = 0.7) +
  labs(title = "Alpha Diversity (Simpson)", y = "Simpson Index") +
  theme_minimal() +
  stat_compare_means(comparisons = comparisons,
                    method = "wilcox.test", # nonparametric test (common for alpha diversity)
                    label = "p.format")

p_alpha_observed <- ggplot(alpha_div, aes(x = Study, y = Observed, fill = Study)) +
  geom_boxplot(width = 0.6, alpha = 0.8) +
  geom_jitter(width = 0.2, size = 1.5, alpha = 0.7) +
  labs(title = "Alpha Diversity (Observed Richness)", y = "Observed Richness") +
  theme_minimal() +
  stat_compare_means(comparisons = comparisons,
                    method = "wilcox.test", # nonparametric test (common for alpha diversity)
                    label = "p.format")

p_alpha_chao <- ggplot(alpha_div, aes(x = Study, y = Chao1, fill = Study)) +
  geom_boxplot(width = 0.6, alpha = 0.8) +
  geom_jitter(width = 0.2, size = 1.5, alpha = 0.7) +
  labs(title = "Alpha Diversity (Chao)", y = "Chao Index") +
  theme_minimal() +
  stat_compare_means(comparisons = comparisons,
                    method = "wilcox.test", # nonparametric test (common for alpha diversity)
                    label = "p.format")

library("cowplot")

# Arrange in 2x2 grid

pad_sig <- function(p, top_mult = 0.22, t = 10, r = 10, b = 10, l = 10){

```

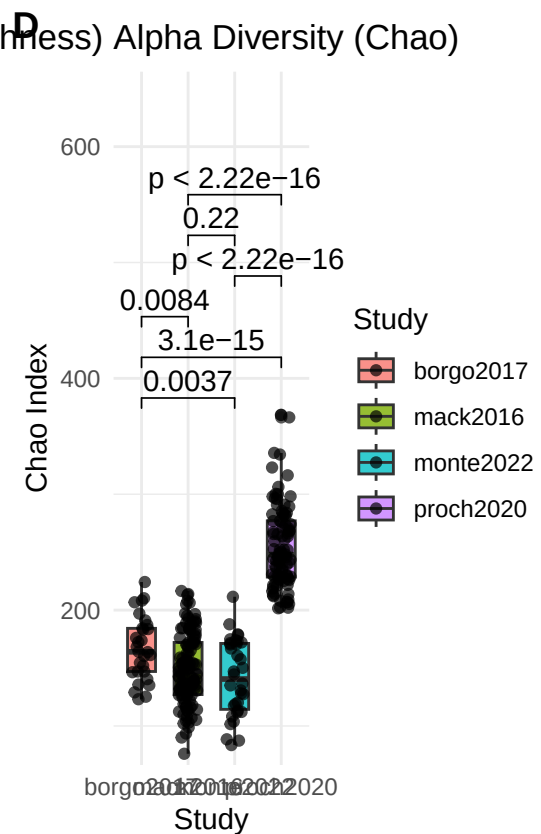
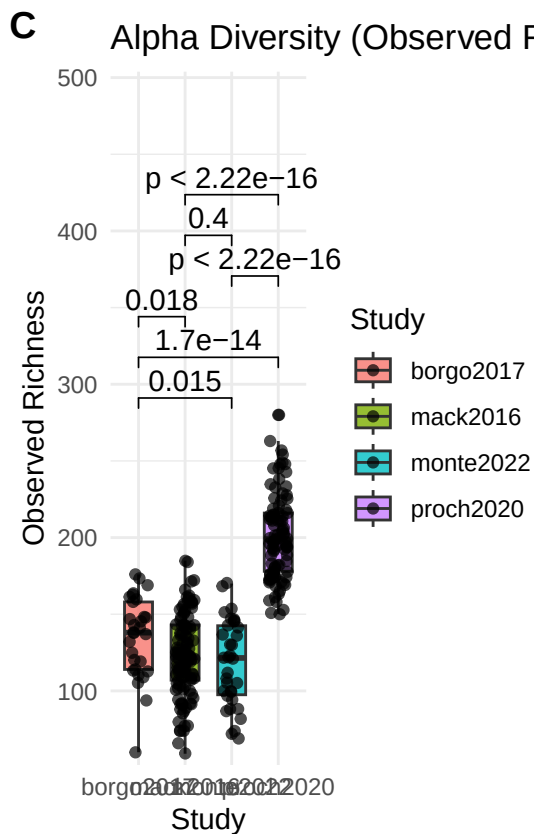
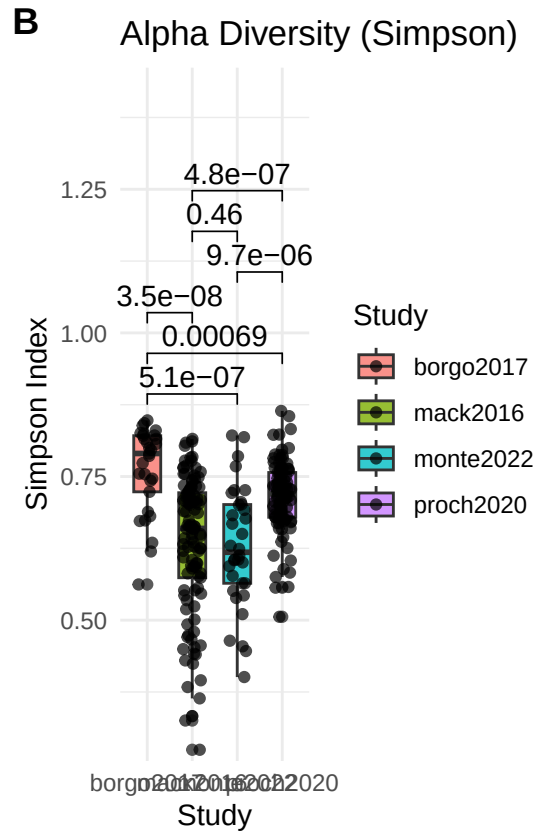
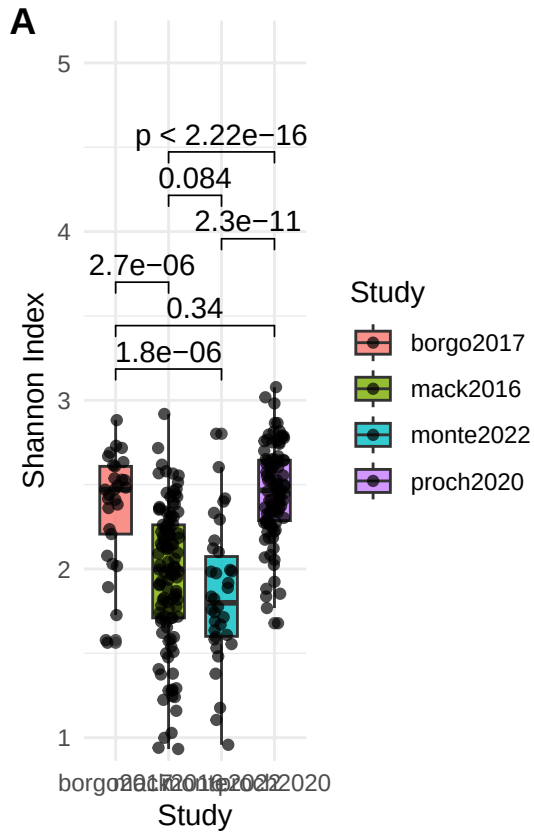
```

p +
  scale_y_continuous(expand = expansion(mult = c(0.02, top_mult))) + # add space above
  coord_cartesian(clip = "off") + # allow drawing outside panel
  theme(plot.margin = margin(t, r, b, l)) # avoid cropping at plot edge
}

sup_fig_2 <- plot_grid(
  pad_sig(p_alpha_shannon), pad_sig(p_alpha_simpson), pad_sig(p_alpha_observed), pad_sig(p_alpha_chao),
  labels = c("A", "B", "C", "D"), # optional labels
  ncol = 2, nrow = 2
)

sup_fig_2

```



```

#plot for category:
p_alpha_shannon_cat <- ggplot(alpha_div, aes(x = Category, y = Shannon, fill = Category)) +
  geom_boxplot(width = 0.6, alpha = 0.8) +
  geom_jitter(width = 0.2, size = 1.5, alpha = 0.7) +
  #labs(title = "Alpha Diversity (Shannon)",
    labs(y = "Shannon Index") +
  theme_minimal() +
  stat_compare_means(comparisons = list(c("ANT1", "HC")),
    method = "wilcox.test", # nonparametric test (common for alpha diversity)
    label = "p.format")

ggsave(
  filename = "shannon_alpha_div_cat.pdf",
  plot = p_alpha_shannon_cat,
  width = 7, # typical 2-column width
  height = 4.5,
  units = "in",
  device = cairo_pdf
)

ggsave(
  filename = "shannon_alpha_div_cat.jpg",
  plot = p_alpha_shannon_cat,
  width = 7,
  height = 4.5,
  units = "in",
  dpi = 600,
  bg = "white"
)

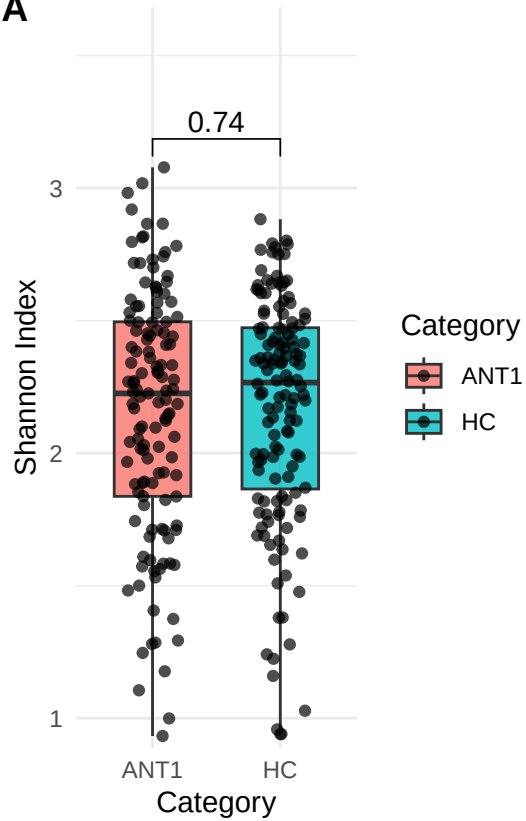
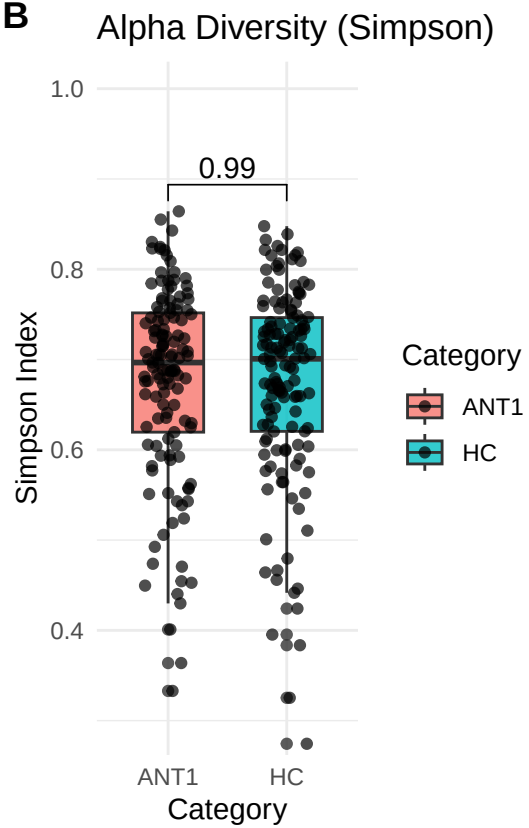
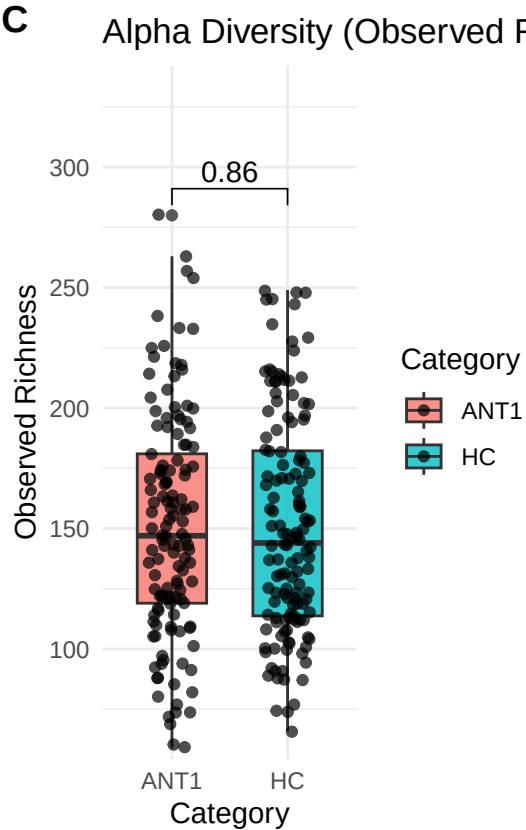
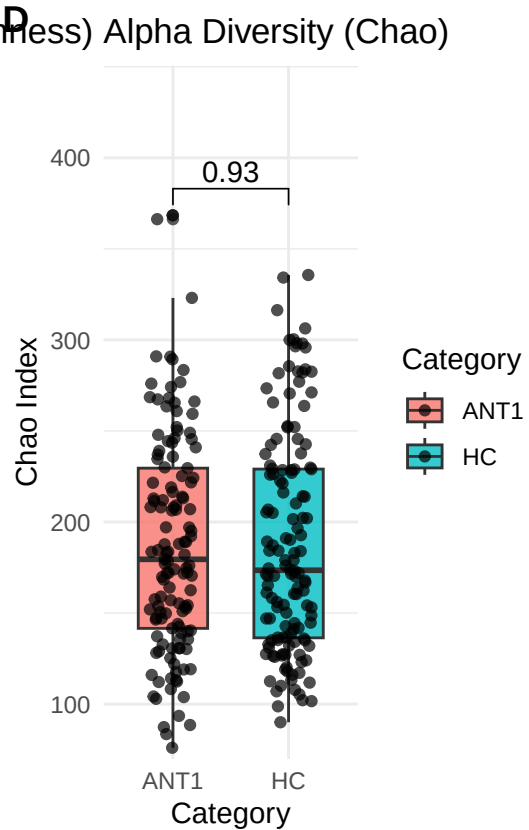
p_alpha_simpson_cat <- ggplot(alpha_div, aes(x = Category, y = Simpson, fill = Category)) +
  geom_boxplot(width = 0.6, alpha = 0.8) +
  geom_jitter(width = 0.2, size = 1.5, alpha = 0.7) +
  labs(title = "Alpha Diversity (Simpson)", y = "Simpson Index") +
  theme_minimal() +
  stat_compare_means(comparisons = list(c("ANT1", "HC")),
    method = "wilcox.test", # nonparametric test (common for alpha diversity)
    label = "p.format")

p_alpha_observed_cat <- ggplot(alpha_div, aes(x = Category, y = Observed, fill = Category)) +
  geom_boxplot(width = 0.6, alpha = 0.8) +
  geom_jitter(width = 0.2, size = 1.5, alpha = 0.7) +
  labs(title = "Alpha Diversity (Observed Richness)", y = "Observed Richness") +
  theme_minimal() +
  stat_compare_means(comparisons = list(c("ANT1", "HC")),
    method = "wilcox.test", # nonparametric test (common for alpha diversity)
    label = "p.format")

p_alpha_chao_cat <- ggplot(alpha_div, aes(x = Category, y = Chao1, fill = Category)) +
  geom_boxplot(width = 0.6, alpha = 0.8) +
  geom_jitter(width = 0.2, size = 1.5, alpha = 0.7) +
  labs(title = "Alpha Diversity (Chao)", y = "Chao Index") +
  theme_minimal() +
  stat_compare_means(comparisons = list(c("ANT1", "HC")),
    method = "wilcox.test", # nonparametric test (common for alpha diversity)
    label = "p.format")

```

```
sup_fig_3 <- plot_grid(  
  pad_sig(p_alpha_shannon_cat), pad_sig(p_alpha_simpson_cat), pad_sig(p_alpha_observed_cat), pad_sig(p_alpha_simpson_cat),  
  labels = c("A", "B", "C", "D"), # optional labels  
  ncol = 2, nrow = 2  
)  
  
sup_fig_3
```

**A****B****C****D**



```

#for singular studies:

# show stars instead of raw p-values

# --- Alpha Diversity Statistics ---
# Kruskal-Wallis
kw_shannon <- kruskal.test(Shannon ~ Study, data = alpha_div)

# Post-hoc Dunn test with FDR correction
dunn_shannon <- dunnTest(Shannon ~ Study, data = alpha_div, method = "bh")

# Kruskal-Wallis for category
kw_shannon_cat <- kruskal.test(Shannon ~ Category, data = alpha_div)

# Post-hoc Wilcoxon tests for category over merged samples
wilcox.test(Shannon ~ Category, data = alpha_div)

##
## Wilcoxon rank sum test with continuity correction
##
## data: Shannon by Category
## W = 8565, p-value = 0.7405
## alternative hypothesis: true location shift is not equal to 0
wilcox.test(Simpson ~ Category, data = alpha_div)

##
## Wilcoxon rank sum test with continuity correction
##
## data: Simpson by Category
## W = 8782, p-value = 0.9878
## alternative hypothesis: true location shift is not equal to 0
wilcox.test(Chao1 ~ Category, data = alpha_div)

##
## Wilcoxon rank sum test with continuity correction
##
## data: Chao1 by Category
## W = 8830, p-value = 0.9265
## alternative hypothesis: true location shift is not equal to 0
wilcox.test(Observed ~ Category, data = alpha_div)

##
## Wilcoxon rank sum test with continuity correction
##
## data: Observed by Category
## W = 8880.5, p-value = 0.8625
## alternative hypothesis: true location shift is not equal to 0
wilcox.test(Shannon ~ Category, data = (alpha_div %>% dplyr::filter(Study == "borgo2017")))

##
## Wilcoxon rank sum exact test
##
## data: Shannon by Category

```

```

## W = 69, p-value = 0.1225
## alternative hypothesis: true location shift is not equal to 0
wilcox.test(Shannon ~ Category, data = (alpha_div %>% dplyr::filter(Study == "mack2016")))

##
## Wilcoxon rank sum test with continuity correction
##
## data: Shannon by Category
## W = 1711, p-value = 0.1717
## alternative hypothesis: true location shift is not equal to 0
wilcox.test(Shannon ~ Category, data = (alpha_div %>% dplyr::filter(Study == "proch2020")))

##
## Wilcoxon rank sum test with continuity correction
##
## data: Shannon by Category
## W = 1190, p-value = 0.3603
## alternative hypothesis: true location shift is not equal to 0
wilcox.test(Shannon ~ Category, data = (alpha_div %>% dplyr::filter(Study == "monte2022")))

##
## Wilcoxon rank sum exact test
##
## data: Shannon by Category
## W = 105, p-value = 0.1816
## alternative hypothesis: true location shift is not equal to 0
wilcox.test(Simpson ~ Category, data = (alpha_div %>% dplyr::filter(Study == "borgo2017")))

##
## Wilcoxon rank sum exact test
##
## data: Simpson by Category
## W = 64, p-value = 0.07692
## alternative hypothesis: true location shift is not equal to 0
wilcox.test(Simpson ~ Category, data = (alpha_div %>% dplyr::filter(Study == "mack2016")))

##
## Wilcoxon rank sum test with continuity correction
##
## data: Simpson by Category
## W = 1669, p-value = 0.2661
## alternative hypothesis: true location shift is not equal to 0
wilcox.test(Simpson ~ Category, data = (alpha_div %>% dplyr::filter(Study == "proch2020")))

##
## Wilcoxon rank sum test with continuity correction
##
## data: Simpson by Category
## W = 1232, p-value = 0.2153
## alternative hypothesis: true location shift is not equal to 0
wilcox.test(Simpson ~ Category, data = (alpha_div %>% dplyr::filter(Study == "monte2022")))

```

```

##
## Wilcoxon rank sum exact test
##
## data: Simpson by Category
## W = 118, p-value = 0.3753
## alternative hypothesis: true location shift is not equal to 0
wilcox.test(Chao1 ~ Category, data = (alpha_div %>% dplyr::filter(Study == "borgo2017")))

##
## Wilcoxon rank sum test with continuity correction
##
## data: Chao1 by Category
## W = 109, p-value = 0.8786
## alternative hypothesis: true location shift is not equal to 0
wilcox.test(Chao1 ~ Category, data = (alpha_div %>% dplyr::filter(Study == "mack2016")))

##
## Wilcoxon rank sum test with continuity correction
##
## data: Chao1 by Category
## W = 1784.5, p-value = 0.06997
## alternative hypothesis: true location shift is not equal to 0
wilcox.test(Chao1 ~ Category, data = (alpha_div %>% dplyr::filter(Study == "proch2020")))

##
## Wilcoxon rank sum test with continuity correction
##
## data: Chao1 by Category
## W = 1061, p-value = 0.9415
## alternative hypothesis: true location shift is not equal to 0
wilcox.test(Chao1 ~ Category, data = (alpha_div %>% dplyr::filter(Study == "monte2022")))

##
## Wilcoxon rank sum exact test
##
## data: Chao1 by Category
## W = 135, p-value = 0.7596
## alternative hypothesis: true location shift is not equal to 0
wilcox.test(Observed ~ Category, data = (alpha_div %>% dplyr::filter(Study == "borgo2017")))

##
## Wilcoxon rank sum test with continuity correction
##
## data: Observed by Category
## W = 105.5, p-value = 1
## alternative hypothesis: true location shift is not equal to 0
wilcox.test(Observed ~ Category, data = (alpha_div %>% dplyr::filter(Study == "mack2016")))

##
## Wilcoxon rank sum test with continuity correction
##
## data: Observed by Category

```

```

## W = 1824, p-value = 0.04018
## alternative hypothesis: true location shift is not equal to 0
wilcox.test(Observed ~ Category, data = (alpha_div %>% dplyr::filter(Study == "proch2020")))

##
## Wilcoxon rank sum test with continuity correction
##
## data: Observed by Category
## W = 1105, p-value = 0.7959
## alternative hypothesis: true location shift is not equal to 0
wilcox.test(Observed ~ Category, data = (alpha_div %>% dplyr::filter(Study == "monte2022")))

##
## Wilcoxon rank sum test with continuity correction
##
## data: Observed by Category
## W = 129, p-value = 0.6052
## alternative hypothesis: true location shift is not equal to 0
# --- Beta Diversity (Bray-Curtis + PCoA) ---
dist_bc <- phyloseq::distance(ps_combined, method = "bray")

ordu <- ordinate(ps_combined, method = "PCoA", distance = dist_bc)

p_beta <- plot_ordination(ps_combined, ordu, color = "Study", shape = "Category") +
  geom_point(size = 3, alpha = 0.8) +
  labs(title = "Beta Diversity (Bray-Curtis PCoA)") +
  theme_minimal()

# --- Beta Diversity Statistics (PERMANOVA) ---
meta_df <- data.frame(sample_data(ps_combined))
adonis_res <- adonis2(dist_bc ~ Study, data = meta_df)

adonis_res_cat <- adonis2(dist_bc ~ Category + Study, data = meta_df)
adonis_res_cat_seq2 <- adonis2(dist_bc ~ Category + Study, data = meta_df, by = "margin")

#Permdisper
# Run PERMDISP for study provenance
groups <- alpha_div$Study
category <- alpha_div$Category
disp <- betadisper(dist_bc, groups)
disp_cat <- betadisper(dist_bc, category)

# Test for significance
permdisp_res <- permutest(disp, permutations = 999)
permdisp_res_cat <- permutest(disp_cat, permutations = 999)

# Results
print(permdisp_res)

##

```

```
## Permutation test for homogeneity of multivariate dispersions
## Permutation: free
## Number of permutations: 999
##
## Response: Distances
##           Df Sum Sq Mean Sq      F N.Perm Pr(>F)
## Groups      3 0.13227 0.044088 8.0372   999 0.001 ***
## Residuals 261 1.43172 0.005486
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

# Optional: pairwise comparisons
TukeyHSD(dis)

## Tukey multiple comparisons of means
## 95% family-wise confidence level
##
## Fit: aov(formula = distances ~ group, data = df)
##
## $group
##           diff           lwr           upr           p adj
## mack2016-borgo2017 -0.05732156 -0.09733517 -0.017307951 0.0014698
## monte2022-borgo2017 -0.04177122 -0.09017868 0.006636232 0.1175648
## proch2020-borgo2017 -0.07522261 -0.11595312 -0.034492095 0.0000177
## monte2022-mack2016 0.01555034 -0.02206767 0.053168342 0.7088136
## proch2020-mack2016 -0.01790105 -0.04493449 0.009132398 0.3193904
## proch2020-monte2022 -0.03345139 -0.07183107 0.004928297 0.1118194

TukeyHSD(dis_cat)

## Tukey multiple comparisons of means
## 95% family-wise confidence level
##
## Fit: aov(formula = distances ~ group, data = df)
##
## $group
##           diff           lwr           upr           p adj
## HC-ANT1 -0.01480742 -0.03287312 0.00325828 0.1077489

# Visualize
# Extract scores for samples and centroids
scores_samp <- as.data.frame(scores(dis, display = "sites"))
scores_samp$Study <- dis$group

scores_cent <- as.data.frame(scores(dis, display = "centroids"))
scores_cent$Study <- rownames(scores_cent)

# Choose custom colors for the four groups
my_cols <- c("borgo2017" = "#d62728", # red
            "monte2021" = "cyan",
            "proch2020" = "#9467bd", # purple
            "mack2016" = "#2ca02c") # green

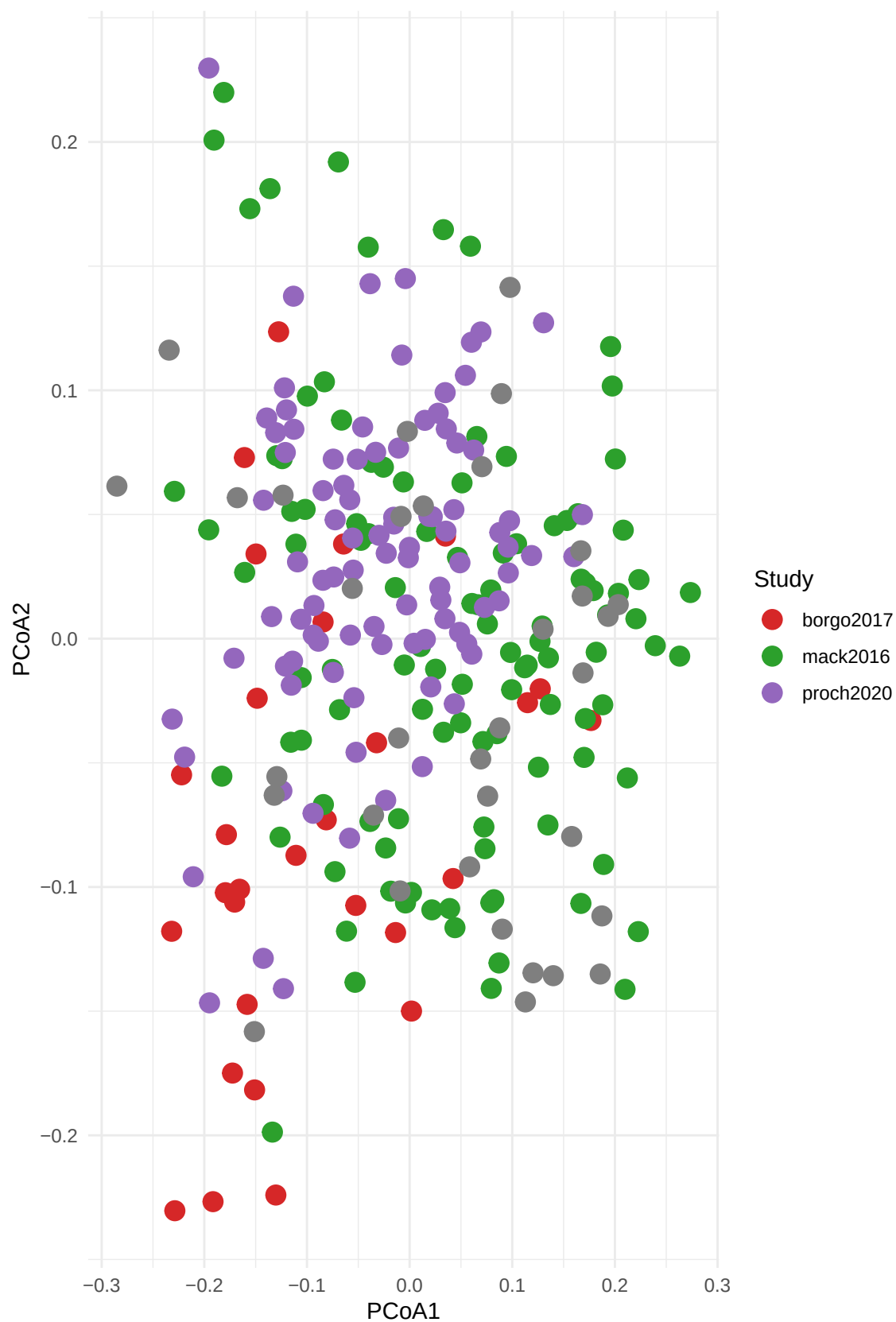
# Build plot
ggplot() +
  geom_point(data = scores_samp,
```

```

#           aes(x = MDS1, y = MDS2, color = Study), alpha = 0.6) +
#stat_ellipse(data = scores_samp,
#           aes(x = MDS1, y = MDS2, color = Study),
#           type = "t", linetype = 2) +
#geom_point(data = scores_cent,
#           aes(x = PCoA1, y = PCoA2, color = Study), size = 4) +
#geom_text_repel(data = scores_cent,
#           aes(x = PCoA1, y = PCoA2, label = Study),
#           size = 5, fontface = "bold") +
scale_color_manual(values = my_cols) +
theme_minimal() +
labs(title = "PERMDISP (beta diversity dispersions)",
     x = "PCoA1", y = "PCoA2")

```

# PERMDISP (beta diversity dispersions)



```

PERMDISP_by_study <- ggplot() +
  geom_point(
    data = scores_samp,
    aes(x = PCoA1, y = PCoA2, color = Study),
    alpha = 0.6,
    size = 2.5
  ) +
  stat_ellipse(
    data = scores_samp,
    aes(x = PCoA1, y = PCoA2, color = Study),
    type = "t",
    linetype = 2,
    linewidth = 0.8
  ) +
  geom_point(
    data = scores_cent,
    aes(x = PCoA1, y = PCoA2, color = Study),
    size = 4
  ) +
  geom_text_repel(
    data = scores_cent,
    aes(x = PCoA1, y = PCoA2, label = Study, color = Study),
    size = 5,
    fontface = "bold",
    show.legend = FALSE
  ) +
  scale_color_manual(values = my_cols) +
  theme_minimal() +
  labs(
    title = "PERMDISP: Bray-Curtis beta diversity by study",
    x = "PCoA1",
    y = "PCoA2",
    color = "Study"
  )
#permdisper boxplot per study:
library(ggplot2)

# Extract distances to centroid
disp_df <- data.frame(
  Study = groups,
  Distance = disp$distances
)

PERMDISP_d_to_centroid_studywise <- ggplot(disp_df, aes(x = Study, y = Distance, color = Study)) +
  geom_boxplot(outlier.shape = NA, linewidth = 0.6) +
  geom_jitter(width = 0.15, alpha = 0.6, size = 2) +
  scale_color_manual(values = my_cols) +
  theme_minimal() +
  labs(
    title = "PERMDISP: Distance to centroid by study",
    x = "Study",

```



```

    y = "Distance to centroid"
  ) +
  guides(color = "none")

#permdisper boxplot per category
disp_df_cat <- data.frame(
  Category = category,
  Distance = disp_cat$distances
)

PERMDISP_d_by_disease <- ggplot(disp_df_cat, aes(x = Category, y = Distance, color = Category)) +
  geom_boxplot(outlier.shape = NA, linewidth = 0.6) +
  geom_jitter(width = 0.15, alpha = 0.6, size = 2) +
  theme_minimal() +
  labs(
    title = "PERMDISP: Distance to centroid by disease group",
    x = "Group",
    y = "Distance to centroid"
  ) +
  guides(color = "none")

#combine into one plot and export:

disp_df <- data.frame(
  Group = groups,
  Distance = disp$distances,
  Type = "Study"
) %>%
  mutate(Group = recode(Group,
    "monte" = "monte2021",
    "mack" = "mack2016",
    "borgo" = "borgo2017",
    "proch" = "proch2020"
  ))

disp_df_cat <- data.frame(
  Group = category,
  Distance = disp_cat$distances,
  Type = "Disease State"
)

combined_df <- bind_rows(disp_df, disp_df_cat)
p <- ggplot(combined_df, aes(x = Group, y = Distance, color = Group)) +
  geom_boxplot(outlier.shape = NA, linewidth = 0.6) +
  geom_jitter(width = 0.15, alpha = 0.6, size = 1.8) +
  facet_wrap(~Type, scales = "free_x") +
  scale_color_manual(values = c(my_cols, "ANT1" = "#E64B35", "HC" = "#4DBBD5")) +
  theme_minimal(base_size = 12) +
  theme(
    strip.text = element_text(face = "bold"),
    axis.text.x = element_text(angle = 30, hjust = 1),

```

```

    legend.position = "none"
  ) +
  labs(
    x = "",
    y = "Distance to centroid"
  )

ggsave(
  filename = "permdisp_combined.pdf",
  plot = p,
  width = 7,          # typical 2-column width
  height = 4.5,
  units = "in",
  device = cairo_pdf
)

ggsave(
  filename = "permdisp_combined.jpg",
  plot = p,
  width = 7,
  height = 4.5,
  units = "in",
  dpi = 600,
  bg = "white"
)

#Species Biplot
#Explanation: A biplot is meant to show which taxa (species/ASVs/genera) are #associated
#with variation in community composition across your full set of samples.
#It answers the question: "What taxa explain the differences between studies?"

#recode study labels:
library(dplyr)
library(ggplot2)
library(ggrepel)
library(grid)

study_labels <- c(
  "monte" = "monte2021",
  "mack" = "mack2016",
  "borgo" = "borgo2017",
  "proch" = "proch2020"
)

study_levels <- c("borgo2017", "mack2016", "monte2021", "proch2020")

#ensure consistent color and theme:
my_cols_pub <- c(
  "borgo2017" = my_cols["borgo2017"],
  "mack2016" = my_cols["mack2016"],
  "monte2021" = my_cols["monte2021"],

```

```

"proch2020" = my_cols["proch2020"]
)

theme_pub <- theme_minimal(base_size = 12) +
  theme(
    panel.grid.minor = element_blank(),
    panel.grid.major = element_line(linewidth = 0.3),
    axis.title = element_text(face = "plain"),
    axis.text = element_text(color = "black"),
    legend.title = element_text(face = "plain"),
    legend.position = "right",
    plot.title = element_text(face = "plain", hjust = 0)
  )

#resolved taxa version with relabeled studies:
resolved_taxa <- taxa_names(ps_combined)[
  !grepl("uncultured|unclassified|not_resolved|NA",
    taxa_names(ps_combined),
    ignore.case = TRUE)
]

ps_resolved <- prune_taxa(resolved_taxa, ps_combined)

#beta div pcoa with relabeled studies:
# Make sure Study is recoded inside the phyloseq object
ps_resolved_recoded <- ps_resolved

sample_data(ps_resolved_recoded)$Study <- recode(
  as.character(sample_data(ps_resolved_recoded)$Study),
  "borgo" = "borgo2017",
  "mack" = "mack2016",
  "monte" = "monte2021",
  "proch" = "proch2020"
)

sample_data(ps_resolved_recoded)$Study <- factor(
  sample_data(ps_resolved_recoded)$Study,
  levels = c("borgo2017", "mack2016", "monte2021", "proch2020")
)

# Re-run ordination after recoding
dist_bc_fr <- phyloseq::distance(ps_resolved_recoded, method = "bray")
ordu_fr <- ordinate(ps_resolved_recoded, method = "PCoA", distance = dist_bc_fr)

# Extract plotting data from phyloseq
ord_df <- plot_ordination(ps_resolved_recoded, ordu_fr, justDF = TRUE)

# Check the column names once:
# names(ord_df)

# Build a vegan ordination object and fitted taxa vectors for the biplot

```

```

ord_vegan_fr <- vegan::wcmdscale(dist_bc_fr, eig = TRUE, add = TRUE)
otu_mat_fr <- as(otu_table(ps_resolved_recoded), "matrix")
if (taxa_are_rows(ps_resolved_recoded)) {
  otu_mat_fr <- t(otu_mat_fr)
}
ef_fr <- vegan::envfit(ord_vegan_fr, otu_mat_fr, permutations = 999)

# Species vectors
scores_species_fr <- as.data.frame(scores(ef_fr, "vectors"))
if (ncol(scores_species_fr) >= 2) {
  names(scores_species_fr)[1:2] <- c("MDS1", "MDS2")
}
scores_species_fr$Taxa <- rownames(scores_species_fr)

# Sample scores
scores_samp_fr <- as.data.frame(scores(ord_vegan_fr, display = "sites"))
if (ncol(scores_samp_fr) >= 2) {
  names(scores_samp_fr)[1:2] <- c("MDS1", "MDS2")
}
scores_samp_fr$Study <- sample_data(ps_resolved_recoded)$Study
scores_samp_fr$Category <- sample_data(ps_resolved_recoded)$Category

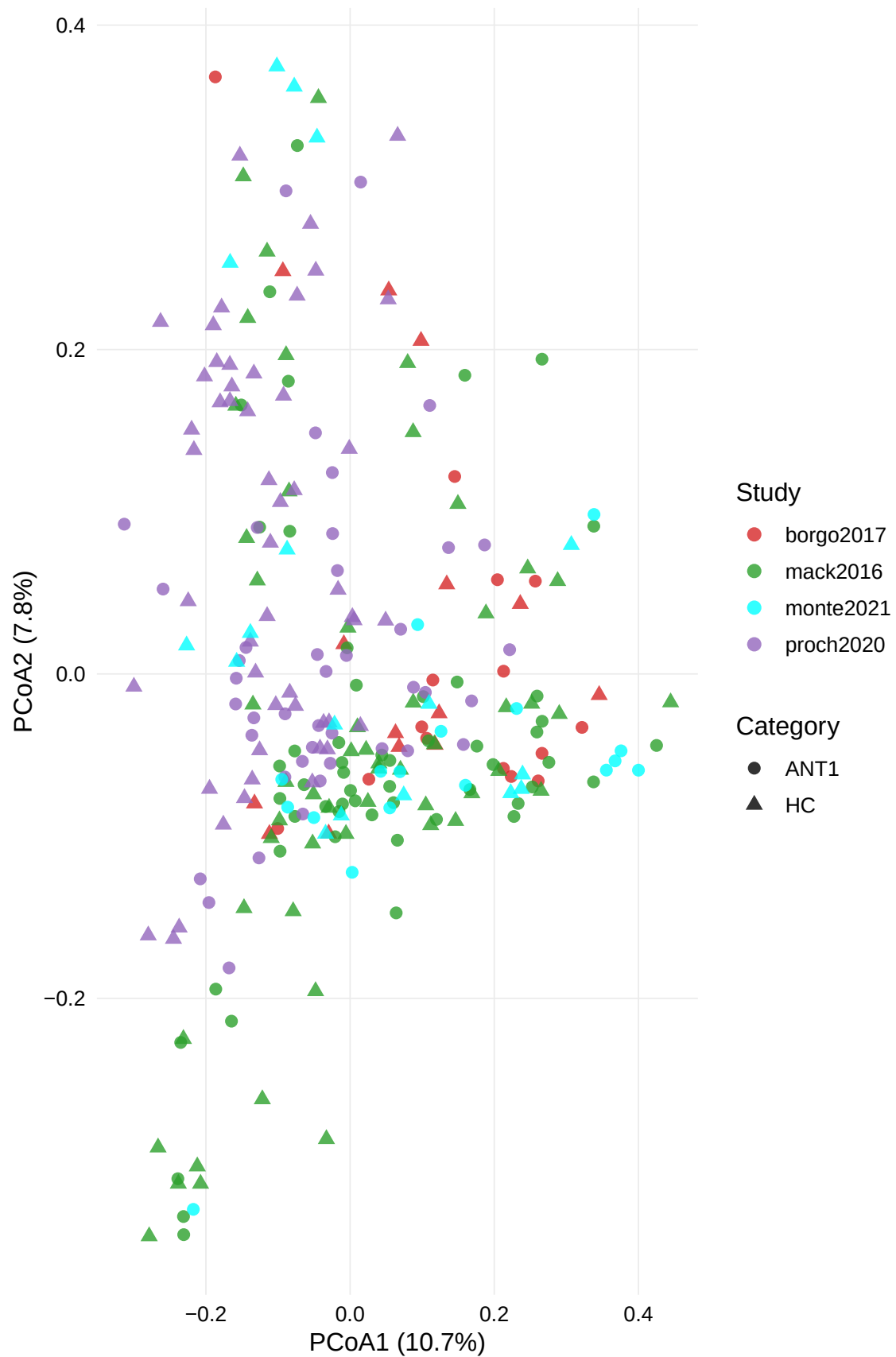
# Percent variance explained
eig_vals <- ordu_fr$values$Eigenvalues
var_exp <- 100 * eig_vals / sum(eig_vals)

xlab_txt <- sprintf("PCoA1 (%.1f%%)", var_exp[1])
ylab_txt <- sprintf("PCoA2 (%.1f%%)", var_exp[2])

# Build plot manually
p_beta_fr <- ggplot(
  ord_df,
  aes(x = Axis.1, y = Axis.2, color = Study, shape = Category)
) +
  geom_point(size = 2.5, alpha = 0.8) +
  scale_color_manual(values = my_cols, drop = FALSE) +
  theme_pub +
  labs(
    x = xlab_txt,
    y = ylab_txt,
    color = "Study",
    shape = "Category"
  )

p_beta_fr

```



```

ggsave(
  "figures/Beta_Div_PCoA.pdf",
  plot = p_beta_fr,
  width = 7, height = 5, units = "in",
  device = cairo_pdf, bg = "white"
)

ggsave(
  "figures/Beta_Div_PCoA.jpg",
  plot = p_beta_fr,
  width = 7, height = 5, units = "in",
  dpi = 600, bg = "white"
)

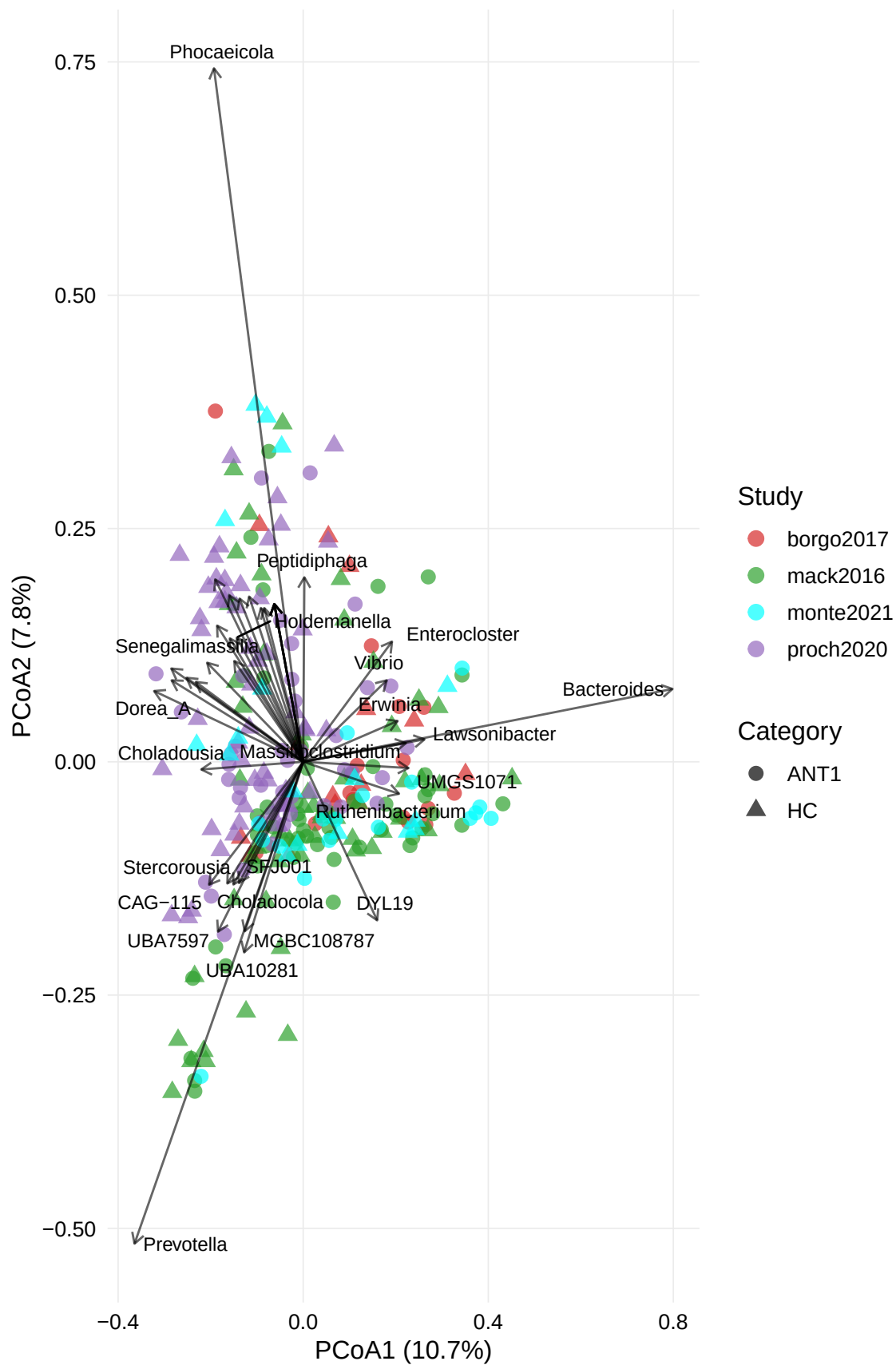
# Add p-values
scores_species_fr$pval <- ef_fr$vectors$pvals

# Keep only significant taxa
scores_species_sig_fr <- scores_species_fr %>%
  filter(pval < 0.01)

species_biplot_sig_pval0_01 <- ggplot(
  scores_samp_fr,
  aes(x = MDS1, y = MDS2, color = Study, shape = Category)
) +
  geom_point(alpha = 0.7, size = 3) +
  geom_segment(
    data = scores_species_sig_fr,
    aes(x = 0, y = 0, xend = MDS1, yend = MDS2),
    inherit.aes = FALSE,
    arrow = arrow(length = unit(0.2, "cm")),
    color = "black",
    alpha = 0.6
  ) +
  geom_text_repel(
    data = scores_species_sig_fr,
    aes(x = MDS1, y = MDS2, label = Taxa),
    inherit.aes = FALSE,
    size = 3,
    color = "black"
  ) +
  scale_color_manual(values = my_cols, drop = FALSE) +
  theme_pub +
  labs(
    x = xlab_txt,
    y = ylab_txt,
    color = "Study",
    shape = "Category"
  )

species_biplot_sig_pval0_01

```



```
ggsave(  
  "figures/PCoA_sig_biplot.pdf",  
  plot = species_biplot_sig_pval0_01,  
  width = 7, height = 5, units = "in",  
  device = cairo_pdf, bg = "white"  
)  
  
ggsave(  
  "figures/PCoA_sig_biplot.jpg",  
  plot = species_biplot_sig_pval0_01,  
  width = 7, height = 5, units = "in",  
  dpi = 600, bg = "white"  
)
```